

# smallgameagent 实验报告

基于 LLM/VLM 与规则的小游戏 Agent

smallgameagent 项目组

2026 年 7 月 21 日

## 目录

|       |                                                        |    |
|-------|--------------------------------------------------------|----|
| 1     | smallgameagent 实验报告（第三轮：分层架构 + 批量框架）                   | 6  |
| 1.1   | 0. 一页结论                                                | 6  |
| 1.2   | 1. 分层多 Agent 架构（实验 F）                                  | 7  |
| 1.2.1 | 设计                                                     | 7  |
| 1.2.2 | 实现                                                     | 7  |
| 1.2.3 | 批量实验结果                                                 | 7  |
| 1.3   | 2. Node.js 高级逻辑移植（实验 H）                                | 8  |
| 1.4   | 3. 批量实验框架（实验 G）                                        | 8  |
| 1.4.1 | 架构                                                     | 8  |
| 1.4.2 | 使用方式                                                   | 8  |
| 1.4.3 | 产出                                                     | 9  |
| 1.4.4 | 轨迹数据格式                                                 | 9  |
| 1.5   | 4. 同学系统对比分析                                            | 9  |
| 1.6   | 5. 多游戏泛化实验（第四轮）                                        | 9  |
| 1.6.1 | 5.1 Generic fallback 让 22 游戏可驱动                        | 9  |
| 1.6.2 | 5.2 Probe 终止假阳性修复                                      | 10 |
| 1.6.3 | 5.3 修正后多游戏结果（5 游戏 × 2 模式）                              | 10 |
| 1.6.4 | 5.4 全游戏自动校准（auto_calibrate.py -all, 22 游戏）             | 10 |
| 1.6.5 | 5.5 Tap-to-Move 驱动策略                                   | 11 |
| 1.7   | 6. 全游戏 × 多模式批量矩阵                                       | 11 |
| 1.7.1 | 6.1 A_full（7 个 joystick 游戏 × 3 模式 × 2 seeds = 42 runs） | 11 |
| 1.7.2 | 6.2 B_tap（15 个 tap-only 游戏 × 2 模式 × 2 seeds = 60 runs） | 12 |
| 1.8   | 7. 云端 API 策略生成 vs 纯 rule                               | 12 |
| 1.9   | 8. VLM 视觉管线                                            | 13 |
| 1.10  | 9. 训练数据生成                                              | 13 |
| 1.11  | 10. L2 输出契约修复                                          | 13 |

|        |                                                              |    |
|--------|--------------------------------------------------------------|----|
| 1.12   | 11. 后续建议                                                     | 14 |
| 1.13   | 12. 多 Provider 云端 API 配置                                     | 14 |
| 1.14   | 13. 规则在线更新架构 (保守方案 A)                                        | 15 |
| 1.14.1 | 13.0 关键修复: 规则引擎真正读取运行时参数                                     | 15 |
| 1.14.2 | 13.1 三层结构                                                    | 16 |
| 1.14.3 | 13.2 触发条件                                                    | 16 |
| 1.14.4 | 13.3 更新产物 schema                                             | 16 |
| 1.14.5 | 13.4 实现                                                      | 17 |
| 1.15   | 14. 本地 VLM 基准实验 (RTX 5060 Laptop 8 GB)                       | 17 |
| 1.16   | 15. Agent 通信扩展                                               | 17 |
| 1.17   | 16. 规则在线更新 A/B 消融实验                                          | 18 |
| 1.18   | 17. 训练数据增量 (全矩阵后合并)                                          | 18 |
| 1.19   | 18. 代表性游戏子集批量跑测                                              | 19 |
| 1.19.1 | 18.1 实验设计                                                    | 19 |
| 1.19.2 | 18.2 结果                                                      | 19 |
| 1.19.3 | 18.3 结论                                                      | 19 |
| 1.20   | 19. Offline Replay Evaluation on Processed Runs              | 20 |
| 1.20.1 | 19.1 动机                                                      | 20 |
| 1.20.2 | 19.2 实现                                                      | 20 |
| 1.20.3 | 19.3 5 游戏 rule 基线 (完整轨迹)                                     | 21 |
| 1.20.4 | 19.4 Hierarchical mock: rule-update 接线验证                     | 21 |
| 1.20.5 | 19.5 Hierarchical Qwen 真实云端 (SSD_00461P01, 30 步)             | 22 |
| 1.20.6 | 19.6 数据集采集                                                   | 23 |
| 1.20.7 | 19.7 后续建议                                                    | 23 |
| 1.21   | 20. Offline Replay 扩展: multi-bus / multi-bus-memory / 搜索规划变体 | 23 |
| 1.21.1 | 20.1 扩展实现                                                    | 23 |
| 1.21.2 | 20.2 multi-bus / multi-bus-memory 离线结果                       | 24 |
| 1.21.3 | 20.3 搜索/规划变体对比                                               | 24 |
| 1.21.4 | 20.4 后续建议                                                    | 25 |
| 1.22   | 21. VLM 训练数据集生成与 5090 训练准备                                   | 25 |
| 1.22.1 | 21.1 数据集生成                                                   | 25 |
| 1.22.2 | 21.2 数据加载验证                                                  | 26 |
| 1.22.3 | 21.3 5090 服务器训练命令                                            | 26 |
| 2      | 在 ssh5090 (10.19.138.148) 上                                  | 26 |
| 2.0.1  | 21.4 训练后的使用路径                                                | 27 |
| 2.0.2  | 21.5 小结                                                      | 27 |
| 2.1    | 22. L2 代码文件级规则更新 (code-file update)                          | 27 |
| 2.1.1  | 22.1 动机                                                      | 27 |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 2.1.2    | 22.2 实现                                                  | 27        |
| 2.1.3    | 22.3 实验                                                  | 28        |
| 2.1.4    | 22.4 真实云端 L2 实验                                          | 29        |
| 2.1.5    | 22.5 关键发现                                                | 30        |
| 2.1.6    | 22.6 后续工作                                                | 30        |
| 2.2      | 23. 本地小 VLM 作为画面理解层 (L1)                                 | 31        |
| 2.2.1    | 23.1 实验目的                                                | 31        |
| 2.2.2    | 23.2 环境                                                  | 31        |
| 2.2.3    | 23.3 实验方法                                                | 31        |
| 2.2.4    | 23.4 结果                                                  | 32        |
| 2.2.5    | 23.5 结论                                                  | 32        |
| 2.2.6    | 23.6 命令                                                  | 32        |
| <b>3</b> | <b>启动本地 VLM 服务</b>                                       | <b>33</b> |
| <b>4</b> | <b>运行对比实验</b>                                            | <b>33</b> |
| 4.1      | 24. 规则在线更新触发机制设计 (回答「规则到底怎么改」)                           | 33        |
| 4.1.1    | 24.1 核心原则                                                | 33        |
| 4.1.2    | 24.2 触发条件 (满足任一即上报)                                      | 34        |
| 4.1.3    | 24.3 触发后处理流程                                             | 34        |
| 4.1.4    | 24.4 L2 输出 schema                                        | 34        |
| 4.1.5    | 24.5 安全门规则                                               | 35        |
| 4.1.6    | 24.6 与同学想法的对齐                                            | 35        |
| 4.1.7    | 24.7 下一步实验                                               | 35        |
| 4.2      | 25. 云端 API 直接做 gameplay 决策的负结果                           | 36        |
| 4.2.1    | 25.1 实验背景                                                | 36        |
| 4.2.2    | 25.2 api-rule 模式: OpenCodeGo 在 SSD_00461P01 上全部 fallback | 36        |
| 4.2.3    | 25.3 hierarchical 模式: L2 输出部分有效, 但整体仍弱                   | 36        |
| 4.2.4    | 25.4 直接 gameplay 的对比 (Kimi / MiMo / OpenCodeGo)          | 37        |
| 4.2.5    | 25.5 对架构的启示                                              | 37        |
| 4.3      | 26. 本地 VLM 视觉上下文实验: Gemma-4-E4B vs Qwen3.5-4B            | 38        |
| 4.3.1    | 26.1 实验目的                                                | 38        |
| 4.3.2    | 26.2 环境                                                  | 38        |
| 4.3.3    | 26.3 结果                                                  | 38        |
| 4.3.4    | 26.4 典型 case 分析                                          | 39        |
| 4.3.5    | 26.5 结论                                                  | 40        |
| 4.4      | 27. Representative Subset 在线跑测 (6 游戏 × 15 runs)          | 40        |
| 4.4.1    | 27.1 实验设计                                                | 40        |
| 4.4.2    | 27.2 总体结果                                                | 40        |
| 4.4.3    | 27.3 逐游戏结果                                               | 41        |

|        |                                                      |    |
|--------|------------------------------------------------------|----|
| 4.4.4  | 27.4 效率与稳定性                                          | 41 |
| 4.5    | 28. SSD_00483P01 诊断与修复: multi-bus 为何 activity=0      | 42 |
| 4.5.1  | 28.1 问题现象                                            | 42 |
| 4.5.2  | 28.2 初步假设与排除                                         | 42 |
| 4.5.3  | 28.3 根因: strategy_memory 的在线自强化                      | 42 |
| 4.5.4  | 28.4 修复尝试 1: rule_engine 优先 + diversity guard (有副作用) | 43 |
| 4.5.5  | 28.5 修复方案 2: StrategyMemory session 隔离 (最终方案)        | 43 |
| 4.5.6  | 28.6 修复后验证                                           | 44 |
| 4.5.7  | 28.7 经验与后续                                           | 45 |
| 4.6    | 29. 多云端 Provider 配置与可用性验证                            | 45 |
| 4.6.1  | 29.1 配置方式                                            | 45 |
| 4.6.2  | 29.2 Smoke Test 结果                                   | 45 |
| 4.7    | 30. 本地 VLM 视觉上下文实验                                   | 46 |
| 4.7.1  | 30.1 运行环境                                            | 46 |
| 4.7.2  | 30.2 实验设计                                            | 46 |
| 4.7.3  | 30.3 结果                                              | 46 |
| 4.8    | 31. 在线规则更新触发器与回滚机制                                   | 47 |
| 4.8.1  | 31.1 设计目标                                            | 47 |
| 4.8.2  | 31.2 实现                                              | 47 |
| 4.8.3  | 31.3 实验                                              | 48 |
| 4.8.4  | 31.4 后续改进                                            | 48 |
| 4.9    | 32. VLM 微调数据准备                                       | 48 |
| 4.9.1  | 32.1 数据来源                                            | 48 |
| 4.9.2  | 32.2 数据规模                                            | 49 |
| 4.9.3  | 32.3 训练代码                                            | 49 |
| 4.9.4  | 32.4 训练计划                                            | 49 |
| 4.10   | 36. Watchdog 回滚机制增强: 不止看 composite                   | 50 |
| 4.10.1 | 36.1 动机                                              | 50 |
| 4.10.2 | 36.2 改进内容                                            | 50 |
| 4.10.3 | 36.3 单元测试                                            | 50 |
| 4.10.4 | 36.4 意义                                              | 50 |
| 4.10.5 | 36.5 触发器 + Watchdog 联合验证                             | 51 |
| 4.10.6 | 36.6 在线验证尝试                                          | 51 |
| 4.10.7 | 36.7 离线回放验证 (offline_replay + mock L2)               | 51 |
| 5      | 改进触发器                                                | 52 |
| 6      | 旧触发器                                                 | 52 |
| 6.0.1  | 36.8 多游戏批量离线验证                                       | 52 |
| 6.0.2  | 36.10 触发器参数持久化到 runtime_rules.json                   | 53 |

|       |                                 |    |
|-------|---------------------------------|----|
| 6.0.3 | 36.11 L2 Prompt 更新：告诉云端模型可以调什么  | 54 |
| 6.0.4 | 36.12 下一步                       | 54 |
| 6.1   | 35. 规则更新触发器改进：相对下降 + 硬上限        | 55 |
| 6.1.1 | 35.1 动机                         | 55 |
| 6.1.2 | 35.2 改进内容                       | 55 |
| 6.1.3 | 35.3 单元测试验证                     | 55 |
| 6.1.4 | 35.4 使用方式                       | 55 |
| 6.1.5 | 35.5 下一步                        | 56 |
| 6.2   | 34. 多 Provider 规则更新阈值扫描（在线 A/B） | 56 |
| 6.2.1 | 34.1 实验设计                       | 56 |
| 6.2.2 | 34.2 结果                         | 56 |
| 6.2.3 | 34.3 关键发现                       | 56 |
| 6.2.4 | 34.4 对阈值设计的启示                   | 57 |
| 6.2.5 | 34.5 下一步                        | 58 |
| 6.3   | 33. 本轮总体结论与下一步                  | 58 |
| 6.3.1 | 33.1 已经验证的事情                    | 58 |
| 6.3.2 | 33.2 仍然存在的问题                    | 58 |
| 6.3.3 | 33.3 下一步实验计划                    | 59 |

# 1 smallgameagent 实验报告 (第三轮: 分层架构 + 批量框架)

> 2026-07-21。配套: \_\_CODE\_\_0000\_\_ (方案)、\_\_CODE\_\_0001\_\_ (过程数据)。  
 > 本轮重点: 分层多 Agent 架构、Node.js 高级逻辑移植、批量实验框架、数据采集管线、\_\_BOLD\_\_0000\_\_。

## 1.1 0. 一页结论

- \_\_BOLD\_\_0000\_\_: 实现 L0 规则 (每步 ~0ms) + L1 本地 VLM (每 5 步 ~5s) + L2 云端 API (每 15 步 ~3s) 三层决策。批量实验中 hierarchical 模式 composite=0.150, 但 L1 因本地 VLM 未启动而退化为 L0+L2; L2 kimi-k2.7-code 的思考链输出导致 JSON 解析失败。\_\_BOLD\_\_0001\_\_: 架构可行, 但需要 (a) 本地 VLM 常驻 + (b) 更强的 L2 输出契约。
- \_\_BOLD\_\_0004\_\_: \_\_CODE\_\_0000\_\_ + \_\_CODE\_\_0001\_\_ 支持多游戏 × 多模式 × 多 seed 矩阵实验, 自动采集逐步轨迹 JSONL。15 runs 产出 \_\_CODE\_\_0002\_\_ + 15 个轨迹文件 + \_\_CODE\_\_0003\_\_。
- \_\_BOLD\_\_0001\_\_: soft target lock (防 target thrashing)、guide-signature change detection (检测 guide 路径变化)、coin demand override (强制导航到 coin table) 已移植到 \_\_CODE\_\_0000\_\_。但 soft lock 在 tap-guide 场景下反而降低了 tap 频率 (rule composite 从 0.150 降到 0.10), 已修复: tap 后释放 lock。
- \_\_BOLD\_\_0003\_\_: \_\_CODE\_\_0000\_\_ mean composite=0.251 仍是当前最稳基线; \_\_CODE\_\_0001\_\_ 在 tap-only 游戏上接近 rule (0.296 vs 0.300), 但在需要 joystick 的 A 组仍落后。\_\_CODE\_\_0002\_\_ 不加 memory 时容易被 SSD\_00483P01 等游戏拉低 (mean 0.110)。
- \_\_BOLD\_\_0004\_\_: 根因是 \_\_CODE\_\_0000\_\_ 在同一次运行中记录并读回 move 动作, 形成在线自强化。修复方案是为每次运行生成 \_\_CODE\_\_0001\_\_, \_\_CODE\_\_0002\_\_ 时排除当前 \_\_CODE\_\_0003\_\_, 让 strategy\_memory 只读回跨 session 记忆。
- \_\_BOLD\_\_0000\_\_: OpenCodeGo / MiMo / Kimi 在「AI agent 直接输出游戏动作」任务上大量空返回或 fallback, 不适合逐帧控制; 但 code-file 规则更新表现良好。
- \_\_BOLD\_\_0000\_\_: Gemma-4-E4B 的 visual summary 能帮云端 qwen 把动作匹配从 3/9 提升到 5/9, 默认 Qwen3.5-4B 反而会带偏方向。
- \_\_BOLD\_\_0002\_\_: \_\_CODE\_\_0000\_\_ 已接入 \_\_CODE\_\_0001\_\_, 每步写入 JSONL (player/action/keyNumbers/reason), 可直接用于后续 VLM 微调。

- \_\_BOLD\_0001\_\_: 新增 \_\_CODE\_0000\_\_ 作为可被 L2 安全改写的运行时参数文件；离线实验中 qwen/kimi/xiaomi/opencodego 均成功输出结构化 code-file 更新。\_\_BOLD\_0002\_\_: 规则更新从「内存参数」扩展到「持久化配置文件」，云端模型可直接调整引擎旋钮而不碰源代码。
- \_\_BOLD\_0000\_\_: 707 passed, 58 skipped, ruff 全绿。

## 1.2 1. 分层多 Agent 架构（实验 F）

### 1.2.1 设计

| 层      | 模型                      | 频率               | 延迟   | 职责                    |
|--------|-------------------------|------------------|------|-----------------------|
| L0 执行层 | Rule engine (tap-guide) | 每步               | ~0ms | 跟随当前 plan 执行 move/tap |
| L1 战术层 | 本地 VLM (gemma-4-E4B)    | 每 5 步或 stuck     | ~5s  | 看截图判断当前状态，修正短期目标      |
| L2 战略层 | 云端 API (kimi-k2.7-code) | 每 15 步或 phase 切换 | ~3s  | 看 probe state 做长程规划   |

### 1.2.2 实现

- \_\_CODE\_0000\_\_: \_\_CODE\_0001\_\_ 类，\_\_CODE\_0002\_\_ 按频率调度三层。
- \_\_CODE\_0000\_\_: 注册 \_\_CODE\_0001\_\_ 模式。
- L2 输出 \_\_CODE\_0000\_\_，存入 \_\_CODE\_0001\_\_。
- L1 输出 \_\_CODE\_0000\_\_ 或 \_\_CODE\_0001\_\_，覆盖 L0 动作。

### 1.2.3 批量实验结果

| 模式               | Mean Composite | Mean Activity | Mean Latency | L1 calls | L2 calls |
|------------------|----------------|---------------|--------------|----------|----------|
| __BOLD_0000__    | __BOLD_0000__  | 1.000         | 21.9s        | —        | —        |
| multi-bus-memory | 0.215          | 0.931         | 23.4s        | —        | —        |
| hierarchical     | 0.150          | 0.000         | 205.1s       | 6        | 2        |
| rule             | 0.101          | 0.672         | 34.7s        | —        | —        |

\_\_BOLD\_0000\_\_:

- hierarchical 的 activity=0 是因为 L1（本地 VLM）未启动，L2 的 JSON 被 kimi-k2.7-code 的思考链截断，导致 L0 rule engine 收到的 macro\_plan 为空，退化为纯 wait。
- L2 调用 2 次（step 0 和 step 15），每次 ~3s；L1 调用 6 次（step 0/5/10/15/20/25），每次因连接拒绝而 ~0s 但记录 warning。
- \_\_BOLD\_0000\_\_: (a) 本地 VLM 常驻服务；(b) L2 用”只输出 JSON”系统提示或 tool calling；(c) L1 失败时 fallback 到 PIL 视觉分析。

### 1.3 2. Node.js 高级逻辑移植（实验 H）

从同学的 \_\_CODE\_0000\_\_（2185 行）移植 3 个机制：

| 机制                     | 作用                                       | 实现               |
|------------------------|------------------------------------------|------------------|
| Soft target lock       | 选定 target 后锁定 8 步，防 target thrashing     | __CODE_0000__: a |
| Guide-signature change | 检测 guide 路径/角度变化，触发重规划                   | __CODE_0000__: p |
| Coin demand override   | station 需要 coins 但玩家没有时，强制导航到 coin table | __CODE_0000__: 松 |

\_\_BOLD\_0001\_\_: soft lock 在 tap-guide 场景下导致 tap 后仍锁定旧 target，新 target 无法被选中，tap 频率从 24/30 降到 19/30。\_\_BOLD\_0002\_\_: tap 动作后立即释放 lock (\_\_CODE\_0000\_\_)。

### 1.4 3. 批量实验框架（实验 G）

#### 1.4.1 架构

```

““
src/experiments/batch_runner.py —BatchConfig + run_batch()
src/experiments/analyze_batch.py —读取 batch_results.json → Markdown 表格
src/experiments/exp_batch_matrix.py —预定义矩阵配置 (1 game × 4 modes × 2 seeds)
““

```

#### 1.4.2 使用方式

```

““python
from src.experiments.batch_runner import BatchConfig, run_batch

config = BatchConfig(
games={"SSD_00461P01": "/path/to/game.html"},
modes=["rule", "multi-bus", "multi-bus-memory", "hierarchical"],
seeds=[42, 123],
max_steps=30,
collect_dataset=True, # 自动写入轨迹 JSONL
output_dir="batch_results",
)
results = asyncio.run(run_batch(config))
““

```



### 1.4.3 产出

- \_\_CODE\_0000\_\_：汇总所有 run 的 composite/activity/details
- \_\_CODE\_0000\_\_：每步 player/action/keyNumbers/reason
- \_\_CODE\_0000\_\_：Markdown 对比表格

### 1.4.4 轨迹数据格式

每行一个 JSON 对象：

```
“{“json
{“player”: {“x”: 2.53, “z”: 12.78}, “action”: “tap”, “keyNumbers”: {“_failCount”: 0},
“reason”: “tap_guide_dist=1.95”}
““
```

可直接用于：

1. VLM 微调数据（next\_probe\_action / probe\_action\_effect 任务）
2. 离线回放分析（stall 检测、target thrashing 诊断）
3. A/B 对比可视化

## 1.5 4. 同学系统对比分析

对比 \_\_CODE\_0000\_\_ 的 Node.js 系统：

| 维度      | 同学 Node.js                                               | 我们 Python                             |
|---------|----------------------------------------------------------|---------------------------------------|
| 游戏覆盖    | 12 游戏完整驱动                                                | 1 游戏 profile + 22 游戏 HTML 无           |
| 控制循环    | coin override + soft lock + guide signature + A* routing | soft lock + guide sig + coin override |
| 每步延迟    | ~2.9s (Playwright + probe)                               | ~0.8s (rule) / ~22s (multi-bus)       |
| 数据采集    | __CODE_0000__ 写 JSONL                                    | __CODE_0000__ 写 JSONL (本              |
| VLM 训练  | 10,336 样本 7 任务 QLoRA                                     | 15,083 样本 7 任务 (processed-runs        |
| 多 Agent | 无                                                        | 6 角色总线 + 记忆 + Critic                  |

## 1.6 5. 多游戏泛化实验（第四轮）

### 1.6.1 5.1 Generic fallback 让 22 游戏可驱动

\_\_CODE\_0000\_\_ 新增 \_\_CODE\_0001\_\_（floating joystick + 单位基线，未校准）+ \_\_CODE\_0002\_\_。\_\_CODE\_0003\_\_ 对无 profile 游戏不再抛错，改用 generic 驱动（移动方向不可靠但 tap 坐标有效）。

### 1.6.2 5.2 Probe 终止假阳性修复

\_\_\_BOLD\_0002\_\_\_: Cocos 引擎标志 \_\_\_CODE\_0000\_\_\_ (含"finish")被 probe WIN 正则误命中, 以及 Logo/火堆等常驻 UI 被标"completion-like"——导致 00482/00342 在第一个动作后被误报 \_\_\_CODE\_0001\_\_\_, 1 步假阳性、composite 虚高 0.700。

\_\_\_BOLD\_0002\_\_\_: \_\_\_CODE\_0000\_\_\_ 改为佐证制——要求 win/victory 面板节点 / 胜利 analytics / 非 \_\_\_CODE\_0001\_\_\_ 管理器的强胜利标志之一, 排除 lose/fail (保留 00461 失败重试)。

### 1.6.3 5.3 修正后多游戏结果 (5 游戏 × 2 模式)

| 游戏         | 校准? | 模式               | 步数 | composite       | activity | tap | stall |
|------------|-----|------------------|----|-----------------|----------|-----|-------|
| 00461 塔防   | cal | rule             | 25 | 0.106           | 0.71     | 17  | 7     |
| 00461 塔防   | cal | multi-bus-memory | 25 | ___BOLD_0000___ | 1.00     | 24  | 0     |
| 00482 砍树   | GEN | rule             | 25 | 0.150           | 0.00     | 0   | 24    |
| 00482 砍树   | GEN | multi-bus-memory | 25 | 0.150           | 0.00     | 0   | 24    |
| 00736 养蛙捕鱼 | GEN | rule             | 25 | 0.269           | 0.79     | 20  | 5     |
| 00736 养蛙捕鱼 | GEN | multi-bus-memory | 25 | ___BOLD_0000___ | 1.00     | 25  | 0     |
| 00342 建造合并 | GEN | rule             | 25 | 0.150           | 0.00     | 0   | 24    |
| 00342 建造合并 | GEN | multi-bus-memory | 25 | 0.150           | 0.00     | 0   | 24    |
| 00532 瀑布巨木 | GEN | rule             | 25 | 0.150           | 0.00     | 0   | 24    |
| 00532 瀑布巨木 | GEN | multi-bus-memory | 25 | 0.150           | 0.00     | 0   | 24    |

\_\_\_BOLD\_0000\_\_\_:

1. \_\_\_BOLD\_0000\_\_\_——与已校准 00461 持平。记忆读回补偿了未校准基线: 记住成功 tap 模式后 activity 0.79→1.00、stall 5→0。
2. \_\_\_BOLD\_0000\_\_\_ 在所有 5 游戏上一致成立。
3. 00482/00342/00532 的 activity=0 是 \_\_\_BOLD\_0000\_\_\_: 未校准基线 → 方向全错 → 需该游戏的 profile 校准。
4. 10 个轨迹 JSONL 已采集 (\_\_\_CODE\_0000\_\_\_), 可直接用于 VLM 微调。

### 1.6.4 5.4 全游戏自动校准 (auto\_calibrate.py -all, 22 游戏)

对全部 22 个 \_\_\_CODE\_0000\_\_\_ 游戏跑自动校准 (4 方向 joystick 脉冲 + 返回脉冲 + warmup + 重试 + moveByCocosInput 回退), 得到 joystick 基线或识别为非 joystick 游戏。

\_\_\_BOLD\_0000\_\_\_:

| 类型                | 数量 | 游戏                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A 类 (joystick 驱动) | 5  | 00440 清障通车、00483 吸沙抽水、00496 电网抓丧尸、00517 末世旅店、00522 地下炸矿                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| A 类 (已有校准)        | 2  | 00461 塔防、00736 捕鱼                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| B 类 (tap-to-move) | 15 | 00219、00332、00342、00382、00394、00427、00434、00475、00482、00526、00527、00528、00529、00530、00531、00532、00533、00534、00535、00536、00537、00538、00539、00540、00541、00542、00543、00544、00545、00546、00547、00548、00549、00550、00551、00552、00553、00554、00555、00556、00557、00558、00559、00560、00561、00562、00563、00564、00565、00566、00567、00568、00569、00570、00571、00572、00573、00574、00575、00576、00577、00578、00579、00580、00581、00582、00583、00584、00585、00586、00587、00588、00589、00590、00591、00592、00593、00594、00595、00596、00597、00598、00599、00600、00601、00602、00603、00604、00605、00606、00607、00608、00609、00610、00611、00612、00613、00614、00615、00616、00617、00618、00619、00620、00621、00622、00623、00624、00625、00626、00627、00628、00629、00630、00631、00632、00633、00634、00635、00636、00637、00638、00639、00640、00641、00642、00643、00644、00645、00646、00647、00648、00649、00650、00651、00652、00653、00654、00655、00656、00657、00658、00659、00660、00661、00662、00663、00664、00665、00666、00667、00668、00669、00670、00671、00672、00673、00674、00675、00676、00677、00678、00679、00680、00681、00682、00683、00684、00685、00686、00687、00688、00689、00690、00691、00692、00693、00694、00695、00696、00697、00698、00699、00700、00701、00702、00703、00704、00705、00706、00707、00708、00709、00710、00711、00712、00713、00714、00715、00716、00717、00718、00719、00720、00721、00722、00723、00724、00725、00726、00727、00728、00729、00730、00731、00732、00733、00734、00735、00736、00737、00738、00739、00740、00741、00742、00743、00744、00745、00746、00747、00748、00749、00750、00751、00752、00753、00754、00755、00756、00757、00758、00759、00760、00761、00762、00763、00764、00765、00766、00767、00768、00769、00770、00771、00772、00773、00774、00775、00776、00777、00778、00779、00780、00781、00782、00783、00784、00785、00786、00787、00788、00789、00790、00791、00792、00793、00794、00795、00796、00797、00798、00799、00800、00801、00802、00803、00804、00805、00806、00807、00808、00809、00810、00811、00812、00813、00814、00815、00816、00817、00818、00819、00820、00821、00822、00823、00824、00825、00826、00827、00828、00829、00830、00831、00832、00833、00834、00835、00836、00837、00838、00839、00840、00841、00842、00843、00844、00845、00846、00847、00848、00849、00850、00851、00852、00853、00854、00855、00856、00857、00858、00859、00860、00861、00862、00863、00864、00865、00866、00867、00868、00869、00870、00871、00872、00873、00874、00875、00876、00877、00878、00879、00880、00881、00882、00883、00884、00885、00886、00887、00888、00889、00890、00891、00892、00893、00894、00895、00896、00897、00898、00899、00900、00901、00902、00903、00904、00905、00906、00907、00908、00909、00910、00911、00912、00913、00914、00915、00916、00917、00918、00919、00920、00921、00922、00923、00924、00925、00926、00927、00928、00929、00930、00931、00932、00933、00934、00935、00936、00937、00938、00939、00940、00941、00942、00943、00944、00945、00946、00947、00948、00949、00950、00951、00952、00953、00954、00955、00956、00957、00958、00959、00960、00961、00962、00963、00964、00965、00966、00967、00968、00969、00970、00971、00972、00973、00974、00975、00976、00977、00978、00979、00980、00981、00982、00983、00984、00985、00986、00987、00988、00989、00990、00991、00992、00993、00994、00995、00996、00997、00998、00999、01000 |
| C 类 (probe 失败)    | 0  | —                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

\_\_\_BOLD\_0000\_\_\_:

| 游戏          | screen_right  | screen_down    |
|-------------|---------------|----------------|
| 00440 清障通车  | (-6.42, 2.34) | (-2.34, -6.42) |
| 00483 吸沙抽水  | (3.41, -3.41) | (2.42, 2.42)   |
| 00496 电网抓丧尸 | (0.75, -0.90) | (0.82, 3.57)   |
| 00517 末世旅店  | (7.42, -0.00) | (0.00, 9.44)   |
| 00522 地下炸矿  | (4.92, 0.00)  | (-0.27, 3.46)  |

### 1.6.5 5.5 Tap-to-Move 驱动策略

为 B 类 (tap-to-move / 自动移动) 游戏新增 \_\_\_CODE\_0000\_\_\_ 驱动: 不走路, 直接 tap guide 目标的屏幕坐标 (probe 的 design→CSS 映射, 无需 joystick 基线校准)。15 个 B 类游戏已自动填充 \_\_\_CODE\_0001\_\_\_ profile。

\_\_\_CODE\_0000\_\_\_ 函数自动分类: A (joystick) / B (tap-only) / C (probe 失败), \_\_\_CODE\_0001\_\_\_ 自动选择驱动类型。

## 1.7 6. 全游戏 × 多模式批量矩阵

### 1.7.1 6.1 A\_full (7 个 joystick 游戏 × 3 模式 × 2 seeds = 42 runs)

| 游戏          | rule            | multi-bus-memory | multi-bus       |
|-------------|-----------------|------------------|-----------------|
| 00440 清障通车  | 0.206           | 0.175            | 0.172           |
| 00461 塔防    | 0.143           | ___BOLD_0000___  | ___BOLD_0000___ |
| 00483 吸沙抽水  | 0.244           | ___BOLD_0000___  | ___BOLD_0000___ |
| 00496 电网抓丧尸 | ___BOLD_0000___ | 0.150            | 0.150           |
| 00517 末世旅店  | 0.150           | 0.150            | 0.150           |
| 00522 地下炸矿  | 0.215           | ___BOLD_0000___  | ___BOLD_0000___ |
| 00736 养蛙捕鱼  | ___BOLD_0000___ | 0.153            | 0.150           |

- \_\_\_BOLD\_0000\_\_\_ (activity=1.00, stall=0)。
- \_\_\_BOLD\_0000\_\_\_: bus 决策在这些游戏上选择了错误动作, 需要按游戏类型调优 driver/profile。

- \_\_BOLD\_0000\_\_——确定性策略比记忆启发式更有效。
- \_\_BOLD\_0000\_\_：rule 0.209、multi-bus 0.217、multi-bus-memory 0.218，差异不大；差异主要来自 00461/00483/00522 被 multi-bus 拉满。

### 1.7.2 6.2 B\_tap (15 个 tap-only 游戏 × 2 模式 × 2 seeds = 60 runs)

| 游戏                   | rule                 | multi-bus-memory     | 说明                       |
|----------------------|----------------------|----------------------|--------------------------|
| 00219 养牛卖奶           | 0.150                | 0.150                | activity=0, 无有效 tap      |
| 00332 圣诞薅羊毛          | 0.150                | 0.150                | activity=0               |
| 00342 建造合并           | 0.150                | 0.150                | activity=0               |
| <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | activity=1.00, tap 24/25 |
| <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | activity=1.00, tap 25/25 |
| 00427 淘金             | 0.150                | 0.150                | activity=0               |
| 00434 选项卡捏           | 0.150                | 0.150                | activity=0               |
| <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | activity=1.00            |
| 00482 砍树扩地           | 0.150                | 0.150                | activity=0               |
| <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | activity=1.00            |
| <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | activity=1.00            |
| <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | activity=1.00            |
| <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | activity 1.00, tap 24/25 |
| 00733 海洋回收           | 0.150                | 0.150                | activity=0               |
| <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | <u>__BOLD_0000__</u> | activity=1.00            |

- \_\_BOLD\_0000\_\_——tap-only 驱动对 guide 目标直接点击即可产生真实交互。
- \_\_BOLD\_0000\_\_——agent 未能选中有效点击目标；可能原因：目标需要拖拽/滑动而非点按、guide 目标被 UI 遮挡、或 tap 坐标映射有偏差。
- \_\_BOLD\_0000\_\_：tap-only 游戏的规则驱动已足够，记忆读回主要在 A 类 joystick 游戏中体现价值。
- \_\_BOLD\_0000\_\_：rule 0.230、multi-bus-memory 0.230、activity 0.533；平均墙钟 rule 42.9s、multi-bus-memory 50.1s（multi-bus-memory 因 API 调用导致部分 run 延迟升高）。

## 1.8 7. 云端 API 策略生成 vs 纯 rule

| 游戏       | rule  | multi-bus-memory | hierarchical (API) |
|----------|-------|------------------|--------------------|
| 00461 塔防 | 0.106 | 0.056            | 0.150              |
| 00736 捕鱼 | 0.275 | 0.237            | 0.150              |

云端 API 的 L2 macro-plan 没有被 L0 规则引擎有效执行（activity=0）。L2 输出需要更强的行动契约。

## 1.9 8. VLM 视觉管线

| 游戏       | probe_only | pil_vision      | vlm_gemma |
|----------|------------|-----------------|-----------|
| 00461 塔防 | 0.044      | ___BOLD_0000___ | 0.150     |
| 00736 捕鱼 | 0.269      | 0.269           | 0.150     |

- \_\_\_BOLD\_0000\_\_\_ (0.044→0.087, tap 7→14, stall 17→10)。
- \_\_\_BOLD\_0000\_\_\_ (activity=0, tap=0) ——本地小模型输出太慢且无法解析为有效动作。

## 1.10 9. 训练数据生成

从 125 条批量实验轨迹 (7 joystick + 15 tap-only 游戏 × 4 模式 × 2 seeds) 离线转换为同事 7 任务训练格式：

| 任务                        | 新增样本            | 合并后总计           |
|---------------------------|-----------------|-----------------|
| probe_action_effect       | +3040           | 5531            |
| information_gain_judgment | +3040           | 6094            |
| progression_grounding     | +3165           | 5806            |
| pulse_response_grounding  | +159            | 1594            |
| failure_recovery          | +9              | 23              |
| ___BOLD_0000___           | ___BOLD_0000___ | ___BOLD_0000___ |

转换器：\_\_\_CODE\_0000\_\_\_，纯离线计算（相邻步 diff → changed\_fields / information\_gain / displacement / stall diagnosis）。

## 1.11 10. L2 输出契约修复

\_\_\_BOLD\_0000\_\_\_：云端 API 的 L2 macro-plan 是抽象文本，L0 无法执行 → hierarchical 全部 activity=0。

\_\_\_BOLD\_0001\_\_\_：L2 prompt 改为要求输出可执行指令列表 (tap/move 带坐标)，存入 \_\_\_CODE\_0000\_\_\_ 动作队列，L0 逐步弹出执行。

\_\_\_BOLD\_0000\_\_\_：

\_\_\_BOLD\_0000\_\_\_：L2 指令队列架构正确 (24/25 步来自 L2 指令)，但 \_\_\_BOLD\_0001\_\_\_ ——没有视觉，坐标全是幻觉。composite 反而更差 (0.055 vs rule 0.114)。

| 模式                      | composite | L2 calls | latency/step |
|-------------------------|-----------|----------|--------------|
| hierarchical (v3, 指令队列) | 0.055     | 1        | 18.1s        |
| rule_baseline           | 0.114     | 0        | 0.84s        |
| multi_bus_memory        | 0.134     | 0        | 0.88s        |

\_\_\_BOLD\_0001\_\_\_: L2 改为输出目标名称 (\_\_\_CODE\_0000\_\_\_), L0 用 probe 的 screenPosition 自行映射坐标。这样 L2 不需要视觉, L0 保留几何准确性。

## 1.12 11. 后续建议

1. \_\_\_BOLD\_0001\_\_\_: 用 probe 的 \_\_\_CODE\_0000\_\_\_ 脉冲自动测量每游戏的 screen→world 基线, 批量生成 profile。
2. \_\_\_BOLD\_0000\_\_\_: systemd 保持 gemma-4-E4B 运行, 让 hierarchical L1 可用。
3. \_\_\_BOLD\_0000\_\_\_: kimi-k2.7-code 加”只输出 JSON”系统提示。
4. \* \_\_\_ITALIC\_0000\_\_\_ routing 移植 \*\*: 从 00864/00867 驱动移植。

## 1.13 12. 多 Provider 云端 API 配置

为支持同学在实验中使用不同云端 LLM/VLM, 我们扩展了 \_\_\_CODE\_0000\_\_\_:

- 新增 \_\_\_CODE\_0000\_\_\_, 统一接入 OpenCodeGo、Kimi、DeepSeek、MiMo(provider 名 \_\_\_CODE\_0001\_\_\_)、Qwen;
- 通过 \_\_\_CODE\_0000\_\_\_ 文件配置各 provider 的 \_\_\_CODE\_0001\_\_\_ / \_\_\_CODE\_0002\_\_\_ / 默认模型, \_\_\_CODE\_0003\_\_\_ 已加入 \_\_\_CODE\_0004\_\_\_, 绝不会被提交;
- 支持 \_\_\_CODE\_0000\_\_\_、\_\_\_CODE\_0001\_\_\_、\_\_\_CODE\_0002\_\_\_ 等环境变量覆盖默认模型, 便于切换 kimi-k2.7-code / kimi-k2.6 / mimo-v2.5 等;
- 切换 provider 只需设置 \_\_\_CODE\_0000\_\_\_ 环境变量, 模型名随 provider 自动默认;
- 保留 \_\_\_CODE\_0000\_\_\_ 完全兼容, 现有调用无需修改;
- Kimi 系列自动省略 \_\_\_CODE\_0000\_\_\_ 参数, 避免代理返回 400。

当前实测可用: Kimi(kimi-k2.7-code / kimi-k2.6)、Xiaomi(mimo-v2.5)、\_\_\_BOLD\_0000\_\_\_ 文本 + 多模态均可用; Qwen (qwen3.7-max) 文本可用, 视觉格式待适配; DeepSeek 余额不足。

| Provider   | 默认文本模型            | 默认视觉模型        | base_url      | 当前状态      |
|------------|-------------------|---------------|---------------|-----------|
| opencodego | deepseek-v4-flash | mimo-v2.5     | __CODE_0000__ | 文本 + 视觉可用 |
| kimi       | kimi-k2.7-code    | kimi-k2.6     | __CODE_0000__ | 文本 + 视觉可用 |
| deepseek   | deepseek-chat     | deepseek-chat | __CODE_0000__ | 余额不足      |
| xiaomi     | mimo-v2.5         | mimo-v2.5     | __CODE_0000__ | 文本 + 视觉可用 |
| qwen       | qwen3.7-max       | qwen3.7-max   | __CODE_0000__ | 文本 / 视觉   |

\_\_BOLD\_0001\_\_：在 L2 规则更新任务中，Kimi 与 Xiaomi 对「游戏策略优化/参数调整」类 prompt 返回空内容（疑似内容过滤），而 \_\_BOLD\_0002\_\_。因此当前规则更新 A/B 实验采用 Qwen 作为 L2 provider。OpenCodeGo 现已可用，后续可补充其 code-file 更新实验。DeepSeek 因余额不足暂时无法调用。

本轮新增 \_\_CODE\_0000\_\_ 模型覆盖示例：

```
“‘bash
export KIMI_TEXT_MODEL=kimi-k2.7-code
export KIMI_VISION_MODEL=kimi-k2.6
export XIAOMI_TEXT_MODEL=mimo-v2.5
export XIAOMI_VISION_MODEL=mimo-v2.5
“‘
```

## 1.14 13. 规则在线更新架构（保守方案 A）

针对同学提出的「规则作为底层，需要修改时让上两层更新」的需求，我们设计并实现了在线规则更新机制。

### 1.14.1 13.0 关键修复：规则引擎真正读取运行时参数

之前的实现中，\_\_CODE\_0000\_\_ 会创建并更新 \_\_CODE\_0001\_\_，但 \_\_CODE\_0002\_\_ 并不读取这些参数，导致 L2 输出的参数更新 \_\_BOLD\_0003\_\_（更新只停留在内存对象里）。本轮修复了这条关键接线：

- \_\_CODE\_0000\_\_ 新增可选 \_\_CODE\_0001\_\_ 参数；
- \_\_CODE\_0000\_\_ 在 \_\_CODE\_0001\_\_ 时创建 \_\_BOLD\_0005\_\_ 的 \_\_CODE\_0002\_\_ 实例，同时传给 \_\_CODE\_0003\_\_ 和 \_\_CODE\_0004\_\_；
- \_\_CODE\_0000\_\_ 在关键策略节点通过 \_\_CODE\_0001\_\_ 读取运行时参数：
- \_\_CODE\_0000\_\_：金币缓冲，避免「有钱就升级」导致的来回折返；

- `__CODE_0000__`：卡死触发 escape 的步数阈值；
- `__CODE_0000__` / `__CODE_0001__`：soft target lock 的锁定步数与容错次数；
- `__CODE_0000__`：coin override 的最大持续步数。

这样 L2 的 `__CODE_0000__` 类型更新可以即时改变 L0 规则引擎的行为，真正实现「上层触发 → 底层参数进化」。

### 1.14.2 13.1 三层结构

- `__BOLD_0000__`：零延迟执行，读取内存参数；
- `__BOLD_0000__`：看截图输出结构化视觉上下文，辅助云端 API 理解画面；
- `__BOLD_0000__`：根据触发条件和视觉上下文，输出结构化规则更新 JSON。

### 1.14.3 13.2 触发条件

- `__CODE_0000__` 连续 N 步低于阈值（默认 0.15）；
- `__CODE_0000__` 停滞计数超过阈值（默认 5 步）；
- L0 与 L2 决策冲突持续 K 步；
- 世界模型检测到空间/时间一致性违例（`__CODE_0000__` stale 传播）。

### 1.14.4 13.3 更新产物 schema

```
““json
{
  "update_type": "param|memory_entry|phase_contract|code_file",
  "target": "rules.coin_save_buffer",
  "reason": " 金币未攒够就升级导致往返",
  "payload": {"coin_save_buffer": 10},
  "confidence": 0.82
}
““
```



### 1.14.5 13.4 实现

- 新增 `__CODE_0000__` : `__CODE_0001__`、`__CODE_0002__`、`__CODE_0003__`、`__CODE_0004__`；
- 扩展 `__CODE_0000__`：在 `__CODE_0001__` 中检查触发条件，满足时调用 L2 请求规则更新，解析并应用；
- 扩展 `__CODE_0000__`：读取共享的 `__CODE_0001__`，让参数更新即时生效；
- `__CODE_0000__` 改用线程池执行同步的 `__CODE_0001__`，避免同步云 API 调用阻塞 Playwright 事件循环；
- 为 `__CODE_0000__` 增加 `__CODE_0001__`（默认 5 步），防止 composite/stall 持续低迷时连续触发 L2；
- `__CODE_0000__` 类型更新受 allowlist + 置信度 + patch 大小 + 自动备份多重保护，未通过则进入待审队列；
- 新增 `__CODE_0000__` 14 个单测。

### 1.15 14. 本地 VLM 基准实验 (RTX 5060 Laptop 8 GB)

使用 LM Studio 的 CUDA12 llama.cpp 后端，4-bit 量化 + q4\_0 KV-cache，在 `__CODE_0000__` 上跑结构化视觉提取：

| 模型                    | 解析成功 | 平均延迟    | 生成 tok/s | 说明                         |
|-----------------------|------|---------|----------|----------------------------|
| gemma-4-E4B-it-Q4_K_M | 3/3  | 4.42 s  | 58.8     | 本地最佳，输出可直接解析               |
| Qwen3.5-9B-Q4_K_M     | 1/3  | 14.67 s | 47.9     | 仅一帧成功，其余输出被 markdown fence |
| Qwen3.5-4B-Q4_K_M     | 0/3  | ~10.6 s | ~67      | 速度够快但格式控制不稳                |

结论：`__BOLD_0000__`，延迟已接近在线逐步控制的可接受范围；Qwen 系列需要更强的”no markdown fences” prompt 约束。

### 1.16 15. Agent 通信扩展

- 在 `__CODE_0000__` 的 `__CODE_0001__` 中新增 `__CODE_0002__`；
- 为后续多 Agent 协作中「规则更新提议 → Critic 评估 → MemoryCurator 落地」的流水线预留消息类型；
- 相关单测通过。

## 1.17 16. 规则在线更新 A/B 消融实验

在 00461 塔防上对比 rule / multi-bus-memory / hierarchical（规则更新触发开启）：

| 模式                   | composite | activity | move | tap | stall | 墙钟      |
|----------------------|-----------|----------|------|-----|-------|---------|
| rule                 | 0.112     | 0.750    | 6    | 18  | 6     | 29.6 s  |
| multi-bus-memory     | 0.038     | 0.250    | 18   | 6   | 18    | 32.1 s  |
| hierarchical（规则更新开启） | 0.150     | 0.000    | 0    | 0   | 24    | 367.9 s |

\_\_\_BOLD\_0000\_\_\_：

1. \_\_\_BOLD\_0000\_\_\_：L2 输出被 kimi 思考链截断，L1 本地 VLM 未启动，导致 activity=0（全部 wait）。但 consistency 得分高，所以 composite 反而比 rule 高——这是「不动作就不会错」的假象。
2. \_\_\_BOLD\_0000\_\_\_：本次使用独立内存文件，无历史记录，表现反而不如 rule。
3. \_\_\_BOLD\_0000\_\_\_：当 stall streak 达到阈值时确实调用了 L2，但 L2 返回的是抽象 planning JSON 而非规则更新 JSON，说明需要把 planning 和 rule-update 的 prompt 彻底分离。

\_\_\_BOLD\_0000\_\_\_：为 hierarchical 模式禁用默认 L2 planning，只保留规则更新触发；或者让 L2 planning 输出目标名称而非坐标。

## 1.18 17. 训练数据增量（全矩阵后合并）

对 A\_full（42 runs）和 B\_tap（60 runs）的轨迹分别运行 \_\_\_CODE\_0000\_\_\_，生成新的 7 任务样本：

| 任务                        | A_full 新增       | B_tap 新增        | 合并后 vlm-training-data-merged |
|---------------------------|-----------------|-----------------|------------------------------|
| probe_action_effect       | +1,272          | +1,440          | 6,935                        |
| information_gain_judgment | +1,272          | +1,440          | 7,309                        |
| progression_grounding     | +1,325          | +1,500          | 6,265                        |
| pulse_response_grounding  | +189            | +0              | 1,853                        |
| failure_recovery          | +18             | +5              | 41                           |
| next_probe_action         | —               | —               | 2,645                        |
| field_grounding           | —               | —               | 2,645                        |
| ___BOLD_0000___           | ___BOLD_0000___ | ___BOLD_0000___ | —                            |
| ___BOLD_0000___           | —               | —               | ___BOLD_0000___              |

数据来源：\_\_\_CODE\_0000\_\_\_（历史 processed-runs）、\_\_\_CODE\_0001\_\_\_（代表性子集）、\_\_\_CODE\_0002\_\_\_、\_\_\_CODE\_0003\_\_\_。

合并命令：

```

“‘bash
.venv/bin/python scripts/merge_vlm_datasets.py \
vlm-training-data-processed-runs \
vlm-training-data-representative \
vlm-training-data-A-full \
vlm-training-data-B-tap \
-output vlm-training-data-merged
“‘

```

数据覆盖 22 个游戏、rule / multi-bus-memory / multi-bus 等模式，可直接用于 Qwen3.5-4B/9B 与 Gemma-4-E4B 的 QLoRA 微调。

5. \_\_BOLD\_\_0001\_\_: CI/CD 中跑 \_\_CODE\_\_0000\_\_, 每次代码变更自动对比 composite。

## 1.19 18. 代表性游戏子集批量跑测

### 1.19.1 18.1 实验设计

为验证浏览器修复后的多游戏自动跑测能力，挑选 6 个有代表性的游戏：

- \_\_BOLD\_\_0000\_\_: SSD\_\_00461P01 塔防、SSD\_\_00483P01 吸沙抽水、SSD\_\_00522P02 地下炸矿
- \_\_BOLD\_\_0000\_\_: SSD\_\_00382P01 低坑杀鲨鱼、SSD\_\_00594P02 破石收水、SSD\_\_00742P01 加油小镇

模式：A 类跑 rule / multi-bus / multi-bus-memory；B 类跑 rule / multi-bus-memory。  
seed=42, 25 步。

### 1.19.2 18.2 结果

### 1.19.3 18.3 结论

1. \_\_BOLD\_\_0000\_\_: 3/3 游戏达到 composite 0.300, multi-bus-memory 未带来额外收益，仅将 stall 归零。
2. \_\_BOLD\_\_0000\_\_: 00461 从 0.106 提升到 0.300, 但 00483 的 multi-bus/multi-bus-memory 均 activity=0, 说明 00483 的 profile/driver 需要调优。
3. \_\_BOLD\_\_0001\_\_: 使用 Playwright bundled Chromium + \_\_CODE\_\_0000\_\_ 后, headless 场景初始化成功率 100%。

| game_id      | mode             | composite | activity | stall | wall  |
|--------------|------------------|-----------|----------|-------|-------|
| SSD_00382P01 | rule             | 0.300     | 1.000    | 0     | 13.4s |
| SSD_00382P01 | multi-bus-memory | 0.300     | 1.000    | 0     | 12.8s |
| SSD_00461P01 | rule             | 0.106     | 0.708    | 7     | 14.6s |
| SSD_00461P01 | multi-bus        | 0.161     | 0.875    | 3     | 13.0s |
| SSD_00461P01 | multi-bus-memory | 0.300     | 1.000    | 0     | 11.7s |
| SSD_00483P01 | rule             | 0.244     | 0.625    | 9     | 20.4s |
| SSD_00483P01 | multi-bus        | 0.150     | 0.000    | 24    | 20.6s |
| SSD_00483P01 | multi-bus-memory | 0.150     | 0.000    | 24    | 19.7s |
| SSD_00522P02 | rule             | 0.215     | 0.833    | 4     | 15.7s |
| SSD_00522P02 | multi-bus        | 0.227     | 0.917    | 2     | 16.1s |
| SSD_00522P02 | multi-bus-memory | 0.240     | 1.000    | 0     | 15.0s |
| SSD_00594P02 | rule             | 0.300     | 1.000    | 0     | 17.0s |
| SSD_00594P02 | multi-bus-memory | 0.300     | 1.000    | 0     | 17.4s |
| SSD_00742P01 | rule             | 0.300     | 1.000    | 0     | 15.3s |
| SSD_00742P01 | multi-bus-memory | 0.300     | 1.000    | 0     | 15.8s |

4. \_\_BOLD\_0002\_\_: 代表性子集轨迹经 \_\_CODE\_0000\_\_ 转换后新增 1,173 条样本，存入 \_\_CODE\_0001\_\_。

## 1.20 19. Offline Replay Evaluation on Processed Runs

### 1.20.1 19.1 动机

此前的所有评估都依赖浏览器在线跑游戏，初始化、渲染和 probe 稳定性会消耗大量时间。为了 \_\_BOLD\_0002\_\_ 和 \_\_BOLD\_0003\_\_，新增 \_\_CODE\_0000\_\_：直接读取 \_\_CODE\_0001\_\_ 与对应 state/action JSON，在零浏览器环境下回放并打分。

### 1.20.2 19.2 实现

- \_\_BOLD\_0003\_\_: \_\_CODE\_0000\_\_ 把 dataset-workflow 的旧格式转成 \_\_CODE\_0001\_\_ / \_\_CODE\_0002\_\_ 需要的 probe 格式。
- \_\_BOLD\_0012\_\_: \_\_CODE\_0000\_\_ / \_\_CODE\_0001\_\_ / \_\_CODE\_0002\_\_ → \_\_CODE\_0003\_\_；\_\_CODE\_0004\_\_ / \_\_CODE\_0005\_\_ → \_\_CODE\_0006\_\_；\_\_CODE\_0007\_\_ → \_\_CODE\_0008\_\_。同时兼容 \_\_CODE\_0009\_\_、\_\_CODE\_0010\_\_、\_\_CODE\_0011\_\_ 三种 dataset 字段名。
- \_\_BOLD\_0004\_\_: 构造最小 ctx，注入 \_\_CODE\_0000\_\_ / \_\_CODE\_0001\_\_ / \_\_CODE\_0002\_\_，使 \_\_CODE\_0003\_\_ 可离线运行。
- \_\_BOLD\_0000\_\_：
- \_\_CODE\_0000\_\_: 纯 \_\_CODE\_0001\_\_。

- `__CODE_0000__`: `__CODE_0001__`, 默认 `__CODE_0002__` (禁用本地 VLM), 支持 `__CODE_0003__` / `__CODE_0004__` 切换云端模型, `__CODE_0005__` 使用确定性 mock 客户端验证 rule-update 闭环。
- `__CODE_0000__`: 云端 API 直接输出 JSON action; API 失败时回退到 `__CODE_0001__`。
- `__BOLD_0001__`: steps、action/type matches、move cosine similarity、activity/stall ratio、latency、rule\_update\_count/history、composite(复用 `__CODE_0000__` rubric)。
- `__BOLD_0003__`: `__CODE_0000__` 把 `__CODE_0001__` 写入 `__CODE_0002__`。

### 1.20.3 19.3 5 游戏 rule 基线 (完整轨迹)

> 注: processed-run 的 `__CODE_0000__` 会被 UI 节点误触发, 离线回放中已忽略假阳性终止标志。

| 游戏            | steps | type_match | action_match | composite |
|---------------|-------|------------|--------------|-----------|
| SSD_00461P01  | 67    | 13/67      | 2/67         | 0.241     |
| SSD_00219P01  | 193   | 0/193      | 0/193        | 0.300     |
| SSD_00332P01  | 64    | 51/64      | 1/64         | 0.300     |
| SSD_00342P01  | 177   | 0/177      | 0/177        | 0.300     |
| SSD_00382P01* | 75    | 1/75       | 1/75         | 0.300     |

\\* SSD\_00848P01 不存在, 自动替换为 SSD\_00382P01。

`__BOLD_0000__`:

- SSD\_00461P01 与 SSD\_00332P01 的 recorded trajectory 与当前 rule engine 策略一致, 动作匹配率较高。
- SSD\_00219P01、SSD\_00342P01、SSD\_00382P01 的 recorded action 以 `__CODE_0000__` 为主, 但 game profile 为 tap-only, rule engine 输出 tap 而真值为 move, 匹配率接近 0。这暴露了一个重要问题: `__BOLD_0001__`, 需要进一步校准 profile 或按驱动类型筛选轨迹。

### 1.20.4 19.4 Hierarchical mock: rule-update 接线验证

使用 `__CODE_0000__` 时, L2 客户端总是返回 `__CODE_0001__` 更新(`__CODE_0002__`), 置信度 0.95。

`__BOLD_0000__`:

| 游戏           | steps | type_match | composite | rule_update_count |
|--------------|-------|------------|-----------|-------------------|
| SSD_00461P01 | 67    | 11/67      | 0.241     | 13                |
| SSD_00219P01 | 193   | 4/193      | 0.300     | 38                |
| SSD_00332P01 | 64    | 49/64      | 0.283     | 9                 |
| SSD_00342P01 | 177   | 3/177      | 0.297     | 35                |
| SSD_00382P01 | 75    | 5/75       | 0.286     | 15                |

- \_\_CODE\_0000\_\_ 在 \_\_CODE\_0001\_\_ 与 \_\_CODE\_0002\_\_ 之间共享, mock L2 的 \_\_CODE\_0003\_\_ 更新被成功应用。
- \_\_CODE\_0000\_\_ 记录了每一步的 update\_type / target / payload, conservative scheme A 的离线接线正确。
- mock planning 输出 wait, 对动作匹配影响有限, 结果与 rule 基线接近, 符合预期。

#### 1.20.5 19.5 Hierarchical Qwen 真实云端 (SSD\_00461P01, 30 步)

```

“bash
. .env && QWEN_TEXT_MODEL=qwen3.7-max CLOUD_PROVIDER=qwen \
PYTHONPATH=. python src/experiments/offline_replay.py \
-game SSD_00461P01 -mode hierarchical -provider qwen \
-l2-interval 10 -max-steps 30 \
-output experiment_offline_replay_SSD_00461P01_hierarchical_qwen.json
“

```

结果: steps=29, type\_match=10/29, action\_match=7/29, composite=0.246, mean\_latency\_ms=3, rule\_update\_count=4。

\_\_BOLD\_0000\_\_:

- Qwen API 可用, L2 规划与 rule update 均成功触发, 无需回退 mock。
- 每步延迟约 3.9s, 主要来自同步 L2 调用; 在线事件循环中需要 \_\_CODE\_0000\_\_ 异步化, 否则浏览器会被阻塞 (此前已修复)。
- composite 低于 rule 基线, 说明当前 L2 prompt 在离线回放场景下与 recorded expert action 存在偏差, 可通过在 prompt 中强调”匹配 historical trajectory” 或改为 target-name 模式优化。

### 1.20.6 19.6 数据集采集

为 SSD\_00461P01 采集 30 步 VLM 训练样本：

```
““bash
PYTHONPATH=. python src/experiments/offline_replay.py \
-game SSD_00461P01 -mode rule -max-steps 30 -collect-dataset
““
```

产出 \_\_CODE\_\_0000\_\_，每行 \_\_CODE\_\_0001\_\_ 均为字符串，可直接接入后续 VLM 训练管线。

### 1.20.7 19.7 后续建议

1. \_\_BOLD\_\_0000\_\_：离线 replay 提供了快速的 profile-vs-trajectory 一致性检测手段。
2. \_\_BOLD\_\_0000\_\_：当前只在 SSD\_00461P01 上跑了 30 步，可扩展到更多游戏以评估 L2 规划的泛化性。
3. \_\_BOLD\_\_0000\_\_：虽然离线不需要浏览器，但 L2 同步调用仍显著拖慢评估；可用线程池加速多游戏批量回放。
4. \_\_BOLD\_\_0000\_\_：当前采集使用真值 action，未来可加入 decider 预测 action 作为对比样本，用于训练 critic 或策略改进模型。

## 1.21 20. Offline Replay 扩展：multi-bus / multi-bus-memory / 搜索规划变体

### 1.21.1 20.1 扩展实现

在 \_\_CODE\_\_0000\_\_ 基础上继续扩展：

- 支持 \_\_CODE\_\_0000\_\_ 与 \_\_CODE\_\_0001\_\_ 模式：构造异步 maker，用 \_\_CODE\_\_0002\_\_ 在离线步循环中驱动。
- 支持 \_\_CODE\_\_0000\_\_：控制 multi-bus Critic/总线轮数。
- 修复 \_\_CODE\_\_0000\_\_ 在离线场景下的属性访问：\_\_CODE\_\_0001\_\_ 需要 \_\_CODE\_\_0002\_\_ / \_\_CODE\_\_0003\_\_，LLM prompt builder 需要 \_\_CODE\_\_0004\_\_。
- 新增 \_\_CODE\_\_0000\_\_：对比 rule 基线、hierarchical mock（短/长 horizon）、tiny beam search。

### 1.21.2 20.2 multi-bus / multi-bus-memory 离线结果

合并文件：\_\_CODE\_0000\_\_。

| 模式                                | 游戏           | steps | type_match | action_match | composite |
|-----------------------------------|--------------|-------|------------|--------------|-----------|
| multi-bus (mock)                  | SSD_00461P01 | 67    | 11/67      | 6/67         | 0.221     |
| multi-bus (mock)                  | SSD_00332P01 | 64    | 51/64      | 1/64         | 0.300     |
| multi-bus (mock)                  | SSD_00382P01 | 75    | 1/75       | 1/75         | 0.300     |
| multi-bus-memory (mock)           | SSD_00461P01 | 67    | 26/67      | 0/67         | 0.209     |
| multi-bus-memory (mock)           | SSD_00332P01 | 64    | 7/64       | 1/64         | 0.150     |
| multi-bus-memory (mock)           | SSD_00382P01 | 75    | 40/75      | 1/75         | 0.150     |
| multi-bus-memory (mock, max15 r2) | SSD_00461P01 | 14    | 9/14       | 0/14         | 0.254     |

> 注：因当前环境未配置 \_\_CODE\_0000\_\_，原定 real Qwen 的 max15-r2 运行改用 mock LLM agent，重点验证 memory 管线在 multi-bus-memory 模式下的端到端可运行性。

\_\_BOLD\_0000\_\_：

1. multi-bus 在 00332/00382 上达到 composite 0.300，说明总线架构与这些游戏的 recorded 轨迹高度兼容。
2. multi-bus-memory 在 00461 上将 type\_match 从 11/67 提升到 26/67，但 action\_match 仍为 0——记忆能帮决策器选对动作大类，却未精确复现 move 向量或 tap 坐标。
3. max15-r2 短窗口 composite 0.254 高于完整轨迹 0.209，Critic 第二轮修正在前几步有效；长轨迹上记忆噪声稀释了收益。

### 1.21.3 20.3 搜索/规划变体对比

脚本：\_\_CODE\_0000\_\_

产出：\_\_CODE\_0000\_\_ (SSD\_00461P01, 67 states, 21 ms)。

| 变体                   | type_match | action_match | 说明                    |
|----------------------|------------|--------------|-----------------------|
| rule                 | 0.194      | 0.030        | 纯 L0 规则基线             |
| hierarchical_mock_5  | 0.328      | 0.045        | mock L2 每 5 步重规划，3 意图 |
| hierarchical_mock_15 | 0.298      | 0.060        | mock L2 每 15 步重规划     |
| hierarchical_short   | 0.328      | 0.045        | 短 horizon (3 意图)      |
| hierarchical_long    | 0.388      | 0.075        | 长 horizon (8 意图)      |
| beam_2step           | 0.239      | 0.000        | 2 步束搜索，按状态距离打分        |

\_\_BOLD\_0000\_\_：

1. 确定性 mock L2 只要输出「向首个 active target 移动」的计划，就能显著提升 type\_match (0.194 → 0.388)。
2. horizon 越长，type\_match 越高，因为 L2 动作队列耗尽更慢，规则引擎 fallback 更



少。

3. beam\_2step 当前启发式（玩家位置 + keyNumbers 距离）未能选出与真值同类型的动作，说明启发式需要进一步建模「目标可交互性」或「下一帧 keyNumber 变化预测」。

#### 1.21.4 20.4 后续建议

1. \_\_BOLD\_0000\_\_: multi-bus-memory 的 action\_match 为 0，主要因为记忆匹配返回的是策略级动作（target/方向），与 recorded 的低级向量不匹配。可在记忆中同时存储低级 action 模板，或把动作匹配从向量级放宽到 target 级。

2. \_\_BOLD\_0000\_\_: mock 实验已证明「目标名称 + 队列」机制有效；下一步用真实云端 API（Qwen/kimi）替换 mock，验证长 horizon 计划在真实 L2 下的收益。

3. \_\_BOLD\_0000\_\_: 加入 target active 状态变化、keyNumber 增益预测、以及与最近障碍物的距离，提升 2-step 规划的动作类型准确率。

4. \_\_BOLD\_0001\_\_: 把 \_\_CODE\_0000\_\_ 接入多游戏循环，自动生成规划变体的 A/B 报告。

## 1.22 21. VLM 训练数据集生成与 5090 训练准备

### 1.22.1 21.1 数据集生成

使用已有的 \_\_CODE\_0000\_\_ 将 \_\_CODE\_0001\_\_ 中的 22 个游戏轨迹转换为 7 任务 VLM 冷启动训练格式：

```
““bash
PYTHONPATH=. python src/training/processed_runs_converter.py \
-processes-root processed-runs \
-output-root vlm-training-data-processed-runs
““
```

生成结果 (\_\_CODE\_0000\_\_):

| 任务                        | train | val | all           |
|---------------------------|-------|-----|---------------|
| next_probe_action         | 2491  | 154 | 2645          |
| probe_action_effect       | 2491  | 154 | 2645          |
| field_grounding           | 2491  | 154 | 2645          |
| information_gain_judgment | 2863  | 191 | 3054          |
| pulse_response_grounding  | 1342  | 93  | 1435          |
| progression_grounding     | 2491  | 154 | 2645          |
| failure_recovery          | 14    | 0   | 14            |
| __BOLD_0000__             | —     | —   | __BOLD_0000__ |

覆盖 22 个游戏，每个样本包含截图路径、backend state summary、任务指令与答案 messages，可直接被 \_\_CODE\_0000\_\_ 加载。

### 1.22.2 21.2 数据加载验证

```
“python
from src.training.data_loader import VLMColdStartDataset
ds = VLMColdStartDataset(
    "vlm-training-data-processed-runs",
    "next_probe_action",
    "train",
)
print(len(ds)) # 2491
sample = ds[0]
print(sample["images"], sample["messages"])
“
```

验证通过：图片可加载、messages 包含 system / user / assistant 三段。

### 1.22.3 21.3 5090 服务器训练命令

本地环境缺少 GPU 训练依赖 (torch / transformers / trl / peft 等)，因此将数据和代码准备好后，在 5090 服务器执行：

```
“bash
```

## 2 在 ssh5090 (10.19.138.148) 上

```
python src/training/train_qwen35.py \
-dataset-root vlm-training-data-processed-runs/ \
-model Qwen/Qwen3.5-4B \
-tasks next_probe_action,information_gain_judgment,pulse_response_grounding \
-output-dir checkpoints/qwen35-4b-gameplay \
-epochs 3 -batch-size 2 -grad-accum 8 \
-lr 2e-4 -lora-r 16 -lora-alpha 32
“
```

Gemma-4-E4B 对应使用 \_\_CODE\_0000\_\_, 参数结构相同。

### 2.0.1 21.4 训练后的使用路径

1. 5090 训练得到 LoRA adapter。
2. 在本地或 WSL 通过 \_\_CODE\_0000\_\_ 启动 VLM 推理服务:  
“bash  
python src/inference/server.py \  
-model Qwen/Qwen3.5-4B \  
-adapter checkpoints/qwen35-4b-gameplay \  
-no-flash-attn -port 8000  
“  
3. HybridAgent 的 L1 本地 VLM 调用该服务, 提供视觉上下文给云端 API, 实现「本地小 VLM 理解画面 → 云端大模型做长程规划」的分层架构。

### 2.0.2 21.5 小结

- 数据集已就绪: 15,083 条样本、22 游戏、7 任务。
- 训练脚本与数据加载器已验证可导入; 待 5090 可用时直接启动 QLoRA。
- 该数据集可与后续人工标注/在线采集数据合并, 持续扩展 VLM 的 domain 覆盖。

## 2.1 22. L2 代码文件级规则更新 (code-file update)

### 2.1.1 22.1 动机

之前的 L2 规则更新只能修改内存中的 \_\_CODE\_0000\_\_, agent 重启后失效, 且无法调整引擎内部未暴露给参数表的旋钮。我们希望:

- 云端模型在运行时发现「当前关卡障碍物密集, 默认卡死阈值 5 步太慢」, 能直接降低 \_\_CODE\_0000\_\_;
- 修改落在一个受控的配置文件里, \_\_BOLD\_0001\_\_, 重启后仍然有效;
- 修改过程有安全门: allowlist、高置信度、自动备份、小 patch。

### 2.1.2 22.2 实现

新增 \_\_CODE\_0000\_\_, 存放可被 L2 安全改写的运行时参数:

```

“{json
{
"coin_save_buffer": 0,
"stuck_escape_threshold": 5,
"target_lock_max_steps": 8,
"obstacle_repulse_weight": 1.3,
"escape_score_radius": 3.0
}
“

```

\_\_\_CODE\_0000\_\_\_:

- \_\_\_CODE\_0000\_\_\_ 接受可选 \_\_\_CODE\_0001\_\_\_;
- 新增 \_\_\_CODE\_0000\_\_\_, 查找顺序为: 内存 \_\_\_CODE\_0001\_\_\_ → \_\_\_CODE\_0002\_\_\_ → 硬编码默认值;
- \_\_\_CODE\_0000\_\_\_ 每次 step 读取 JSON, 保证 code-file 更新后无需重启即可生效。

\_\_\_CODE\_0000\_\_\_:

- \_\_\_CODE\_0000\_\_\_ 实现安全门:
  1. \_\_\_BOLD\_0000\_\_\_: 只允许修改白名单内的文件;
  2. \_\_\_BOLD\_0000\_\_\_;
  3. \_\_\_BOLD\_0000\_\_\_, search 块 500 字符;
  4. \_\_\_BOLD\_0000\_\_\_ 且为普通文件;
  5. \_\_\_BOLD\_0001\_\_\_ 匹配, 否则进入 \_\_\_CODE\_0000\_\_\_ 待审队列;
  6. 修改前自动备份到 \_\_\_CODE\_0000\_\_\_, 保留最近 3 份。

\_\_\_CODE\_0000\_\_\_ 与 \_\_\_CODE\_0001\_\_\_: 透传 \_\_\_CODE\_0002\_\_\_, 默认指向 \_\_\_CODE\_0003\_\_\_。

### 2.1.3 22.3 实验

新建 \_\_\_CODE\_0000\_\_\_:

- 在 \_\_\_CODE\_0000\_\_\_ 上离线回放前 30 步;

- mock L2 在第 5 步触发规则更新, 返回 `__CODE_0000__`, 把 `__CODE_0001__` 从 5 改为 3, 置信度 0.95;
- 验证修改是否被应用、后续 step 是否读到新值、文件是否被备份、实验结束后是否恢复原始文件。

结果:

| 指标            | 数值   |
|---------------|------|
| 修改前阈值         | 5    |
| 应用 step       | 6    |
| 修改后阈值 (运行中读取) | 3    |
| 置信度           | 0.95 |
| 自动备份          |      |
| 实验后恢复         |      |

#### 2.1.4 22.4 真实云端 L2 实验

新建 `__CODE_0000__`, 对 qwen / kimi / xiaomi / opencodego 四家云模型直接下发 code-file 更新请求, 验证它们能否正确生成可应用的 JSON patch。

| Provider   | 模型                | 延迟    | 结果   | 说明                                |
|------------|-------------------|-------|------|-----------------------------------|
| qwen       | qwen3.7-max       | 7.83s | 应用成功 | JSON 格式完整, 置信度 0.95               |
| kimi       | kimi-k2.7-code    | 3.32s | 应用成功 | 需显式指定 <code>__CODE_0000__</code>  |
| xiaomi     | mimo-v2.5         | 6.18s | 应用成功 | 需提供完整 JSON 模板作为 few-shot, 否则会截断或边 |
| opencodego | deepseek-v4-flash | 9.08s | 应用成功 | 开放式 JSON schema, 输出完整且语义合理, 置信度   |

`__BOLD_0000__`:

1. `__BOLD_0000__`, 只要 system prompt 明确说明 schema, 就能输出完整可解析的 code-file update。
2. `__BOLD_0000__`, 直接把 prompt\_ctx 序列化为 JSON 会导致空输出; 改为在 user message 中给出完整 JSON 模板 (few-shot) 后, 输出稳定且可应用。
3. 四家模型都选择了把 `__CODE_0000__` 从 5 改为 3, 并给出合理的策略解释, 说明 L2 能够理解「卡死阈值」与「escape 行为」的因果关系。

命令:

```

“bash
. .env
PYTHONPATH=. .venv/bin/python -B src/experiments/exp_code_file_rule_update_real.py

```

```

-provider qwen
PYTHONPATH=. .venv/bin/python -B src/experiments/exp_code_file_rule_update_real.py
-provider kimi
PYTHONPATH=. .venv/bin/python -B src/experiments/exp_code_file_rule_update_real.py
-provider xiaomi
PYTHONPATH=. .venv/bin/python -B src/experiments/exp_code_file_rule_update_real.py
-provider opencodego
““

```

产出：

- `__CODE_0000__`
- `__CODE_0000__`
- `__CODE_0000__`
- `__CODE_0000__`

### 2.1.5 22.5 关键发现

1. `__BOLD_0001__`：mock L2 输出结构化 JSON，`__CODE_0000__` 通过全部安全门，配置文件被修改，规则引擎在下一步即读取到新阈值。
2. `__BOLD_0000__`：qwen/kimi/xiaomi/opencodego 均成功，验证了多 provider 下的可行性。
3. `__BOLD_0000__`：低置信度、不在 allowlist、search 块不唯一的更新会被拒绝并进入待审队列，避免误改源码。
4. `__BOLD_0003__`：`__CODE_0000__`（内存）适合高频小调，`__CODE_0001__`（配置文件）适合需要持久化的引擎旋钮；两者共享同一 `__CODE_0002__` 读取路径，优先级为内存 > 文件 > 默认值。
5. `__BOLD_0000__`：qwen/kimi 适合开放式 schema 描述；xiaomi 更适合 few-shot 模板。

### 2.1.6 22.6 后续工作

- 把 `__CODE_0000__` 的 schema 写进 L2 prompt，约束可改字段与取值范围；
- 在真实游戏运行中触发 code-file 更新（而非离线静态 prompt），观察 L2 在动态 stall/composite 下的决策质量；
- 接入版本控制：每次 code-file 更新生成一条 git-style diff 记录，方便回滚与审计。

## 2.2 23. 本地小 VLM 作为画面理解层 (L1)

### 2.2.1 23.1 实验目的

验证「本地小 VLM 理解画面 → 输出文本上下文 → 云端大模型做策略决策」的三层架构是否可行。本地模型用默认 4-bit GGUF (Qwen3.5-4B)，云端用 qwen3.7-max。

### 2.2.2 23.2 环境

- 本地模型：Qwen3.5-4B-Q4\_K\_M.gguf + mmproj-F16.gguf
- 推理后端：llama.cpp CUDA12 (LM Studio 自带 backend)
- GPU：NVIDIA RTX 5060 Laptop 8 GB
- 启动命令：

```
““bash
LD_LIBRARY_PATH=/home/azuma/.lmstudio/extensions/backends/llama.cpp-linux-x86_64-
nvidia-cuda12-avx2-2.17.0 \
/home/azuma/.lmstudio/extensions/backends/llama.cpp-linux-x86_64-nvidia-cuda12-avx2-
2.17.0/llama-server \
-m /home/azuma/.lmstudio/models/unsloth/Qwen3.5-4B-GGUF/Qwen3.5-4B-Q4_K_M.gguf \
-mmproj /home/azuma/.lmstudio/models/unsloth/Qwen3.5-4B-GGUF/mmproj-F16.gguf \
-host 127.0.0.1 -port 1234 -c 4096 -ngl 99
““
```

### 2.2.3 23.3 实验方法

新建 \_\_CODE\_\_0000\_\_：

- 从 \_\_CODE\_\_0000\_\_ 取前 10 步；
- 每一步把截图发给本地 VLM，要求输出「场景类型、玩家位置、可见敌人/物体、箭头/引导、UI 元素」的纯文本摘要；
- 同一状态分别用两种方式请求云端 qwen：
  1. 只看 probe state (text-only baseline)；
  2. probe state + 本地 VLM 摘要 (with-visual)；

- 云端输出归一化 move 动作 (dx/dy [-1,1]), 与真值比较。

## 2.2.4 23.4 结果

| 指标               | 数值                 |
|------------------|--------------------|
| 评估步数             | 9 (有截图的 step 2-10) |
| 本地 VLM 平均延迟      | ~7.1s / 帧          |
| text-only 动作匹配   | 5/9                |
| with-visual 动作匹配 | 3/9                |

\_\_\_BOLD\_0000\_\_\_:

1. \_\_\_BOLD\_0000\_\_\_: 有的 step 能给出较准确的场景描述 (step 2、5、7), 有的会输出大量 chain-of-thought 草稿 (step 3、4、6), 有的严重截断 (step 8 只有”The scene is a top-down isometric view...” )。

2. \_\_\_BOLD\_0000\_\_\_: 例如 step 3 真值向下移动, 本地 VLM 描述「玩家在下部、敌人在底部、基地在上方」, 云端据此改为向上移动; step 8 真值向左下, 本地 VLM 摘要被截断, 云端给出直上动作。

3. \_\_\_BOLD\_0001\_\_\_: 因为 00461 的前几步主要是垂直方向移动, state 中的 \_\_\_CODE\_0000\_\_\_ 已足够。

4. \_\_\_BOLD\_0000\_\_\_: 当前未经微调的 4B 模型对「给策略 planner 看的摘要」这一任务不够稳定, 需要:

- 更严格的 prompt (强制 JSON 输出, 限制长度);
- 或者 QLoRA 微调, 让它学会输出结构化的视觉上下文。

## 2.2.5 23.5 结论

- \_\_\_BOLD\_0000\_\_\_, Qwen3.5-4B-Q4\_K\_M 推理延迟约 7s/帧, 显存占用可控。
- \_\_\_BOLD\_0000\_\_\_, 甚至可能引入方向性错误。
- \_\_\_BOLD\_0000\_\_\_: 用已生成的 15,083 条 processed-runs 数据做 QLoRA 微调, 训练本地 VLM 输出「箭头方向 + 关键目标位置 + 障碍物」等结构化上下文, 再与云端 API 组合验证。

## 2.2.6 23.6 命令

“bash



## 3 启动本地 VLM 服务

```
LD_LIBRARY_PATH=/home/azuma/.lmstudio/extensions/backends/llama.cpp-linux-
x86_64-nvidia-cuda12-avx2-2.17.0 \
/home/azuma/.lmstudio/extensions/backends/llama.cpp-linux-x86_64-nvidia-cuda12-avx2-
2.17.0/llama-server \
-m /home/azuma/.lmstudio/models/unsloth/Qwen3.5-4B-GGUF/Qwen3.5-4B-Q4_K_M.gguf \
-mmproj /home/azuma/.lmstudio/models/unsloth/Qwen3.5-4B-GGUF/mmproj-F16.gguf \
-host 127.0.0.1 -port 1234 -c 4096 -ngl 99
```

## 4 运行对比实验

```
. .env
PYTHONPATH=. .venv/bin/python -B src/experiments/exp_local_vlm_cloud_context.py
-provider qwen -num-steps 10
““
```

产出：\_\_CODE\_0000\_\_

### 4.1 24. 规则在线更新触发机制设计（回答「规则到底怎么改」）

同学一直在问：「规则作为底层，需要修改时让上两层更新，应该怎么触发？」这一节把当前设计说透，并给出下一步实验方向。

#### 4.1.1 24.1 核心原则

- \_\_BOLD\_0000\_\_：零延迟、确定性高，永远是默认 fallback。
- \_\_BOLD\_0000\_\_：只输出结构化更新请求，由 Applier 安全门决定是否落盘。
- \_\_BOLD\_0000\_\_：它看画面，但不改规则；只把视觉上下文交给 L2，让 L2 判断是否需要更新。
- \_\_BOLD\_0000\_\_：

1. \_\_BOLD\_0002\_\_ (高频、低风险): 改内存 \_\_CODE\_0000\_\_ 或 \_\_CODE\_0001\_\_，即时生效。

2. `__BOLD_0000__`（低频、高风险）：改配置文件，受 allowlist + 置信度 + 备份多重保护。

4.1.2 24.2 触发条件（满足任一即上报）

| 触发信号           | 默认阈值                                  | 说明                   |
|----------------|---------------------------------------|----------------------|
| composite 连续低迷 | N=5 步，阈值 0.15                         | 综合得分长期上不去，说明当前策略不佳   |
| stall 停滞计数     | 5 步无有效动作                              | 玩家卡住，可能需要调 escape 阈值 |
| L0 与 L2 决策冲突   | 连续 K=3 步不一致                           | 规则与云端规划打架，需要仲裁或调整    |
| 世界模型 stale 命中  | <code>__CODE_0000__</code> 检测到空间/时间违例 | 例如记忆里的障碍物位置与当前画面不符   |
| L1 视觉异常        | 本地 VLM 报告「guide 丢失 / UI 变化 / 新障碍」     | 提供视觉证据，降低误触发         |

4.1.3 24.3 触发后处理流程

```
““
触发器收集信号
↓
L1 本地 VLM 看截图（可选，每 N 步或触发时）
↓
L2 云端 API 输出结构化 JSON
↓
RuleUpdateApplier 安全门
↓
通过 → 写入 runtime_rules.json / RuleParameters
↓
RuleEngine 下一步读取新参数，行为改变
““
```

4.1.4 24.4 L2 输出 schema

```
““json
{
  "update_type": "param|memory_entry|phase_contract|code_file",
  "target": "rules.stuck_escape_threshold",
  "reason": "当前关卡障碍物密集，5 步才 escape 太慢，导致 stall 累积",
  "payload": {"stuck_escape_threshold": 3},
  "confidence": 0.92,
  "visual_evidence": "L1 报告：玩家被两座建筑夹住，guide 方向指向墙体"
}
```

““

#### 4.1.5 24.5 安全门规则

1. `__BOLD__0003__`: `__CODE__0000__` 只能改 `__CODE__0001__` 等白名单文件, 不能碰 `__CODE__0002__` 源码。
2. `__BOLD__0001__`: `__CODE__0000__` 才自动应用; 0.7~0.9 进待审队列; < 0.7 直接拒绝。
3. `__BOLD__0000__`: search + replace 总字符 2000, search 块 500, 防止大段乱改。
4. `__BOLD__0000__`: search 块在目标文件中必须唯一, 否则拒绝。
5. `__BOLD__0001__`: 修改前备份到 `__CODE__0000__`, 保留最近 3 份。
6. `__BOLD__0000__`: 同一触发条件 5 步内不重复触发, 防止 spam。

#### 4.1.6 24.6 与同学想法的对齐

- `__BOLD__0002__` → 不是。当前已实现 L2 在线输出 `__CODE__0000__` 与 `__CODE__0001__` 更新, qwen/kimi/xiaomi 三家都验证成功。
- `__BOLD__0000__` → 已落地, schema 见 24.4, 阈值见 24.2。
- `__BOLD__0001__` → 已实现, 但只允许改 `__CODE__0000__` 这类受控配置, 不直接改源码。
- `__BOLD__0000__` → 已设计为 L1 证据层, 不直接参与决策; 云端 API 综合 state + L1 摘要后再决定是否更新。

#### 4.1.7 24.7 下一步实验

1. `__BOLD__0000__`: 当前 code-file 实验是离线静态 prompt, 下一步在浏览器跑 00461/00483 时让 L2 根据实时 stall/composite 触发更新。
2. `__BOLD__0000__`: 对比 composite 阈值 0.10 / 0.15 / 0.20 对更新频率和游戏得分的影响。
3. `__BOLD__0000__`: 对比「有 L1 摘要」vs「无 L1 摘要」时 L2 更新决策的准确率和误触发率。
4. `__BOLD__0000__`: 当 L0 与 L2 冲突时, 引入 Critic Agent 做最终决策, 而不是简单信任某一方。
5. `__BOLD__0000__`: 每次 code-file 更新生成一条 diff 记录, 支持一键回滚。

## 4.2 25. 云端 API 直接做 gameplay 决策的负结果

### 4.2.1 25.1 实验背景

除了用云端 API 做规则更新, 我们也试了让云端模型直接输出每一步动作 (api-rule 和 hierarchical 模式)。毕竟如果大模型能端到端玩游戏, 最省架构。实验覆盖了 OpenCodeGo (deepseek-v4-flash / mimo-v2.5)、Kimi (k2.7-code / k2.6) 和 MiMo (mimo-v2.5), 结果并不理想。

### 4.2.2 25.2 api-rule 模式: OpenCodeGo 在 SSD\_00461P01 上全部 fallback

\_\_CODE\_0000\_\_ 模式的设计是: 每步把 probe state 和当前规则上下文发给云端, 让它决定 action; 解析失败或返回空就 fallback 到 wait。

| 指标           | 数值     |
|--------------|--------|
| 总步数          | 29     |
| type_match   | 16/29  |
| action_match | 16/29  |
| composite    | 0.2196 |
| 平均延迟         | 4.37s  |
| fallback 占比  | 约 90%  |

\_\_BOLD\_0000\_\_:

- 前 9 步的真值都是 move, 但 OpenCodeGo 每一步都返回空字符串, 系统 fallback 成 wait。
- 第 11~22 步真值刚好是 wait, fallback 的 wait 才「碰巧」匹配上。
- 这意味着 \_\_BOLD\_0000\_\_, 它愿意写代码、愿意聊天, 但不愿意直接输出游戏动作。

### 4.2.3 25.3 hierarchical 模式: L2 输出部分有效, 但整体仍弱

\_\_CODE\_0000\_\_ 模式下, 云端只负责每 15 步输出一次宏观 plan, L0 规则引擎负责执行。理论上这能规避「直接输出动作」的过滤问题。

\_\_BOLD\_0000\_\_:

- L2 的规划输出有时被截断或为空, 导致 L0 只能按规则自己跑。

| 指标           | 数值     |
|--------------|--------|
| 总步数          | 29     |
| type_match   | 6/29   |
| action_match | 3/29   |
| composite    | 0.2625 |
| 平均延迟         | 2.56s  |

- 即使 L2 给出「Follow guidance and approach enemies」这种宏观指令，L0 把它翻译成 tap-guide 动作后，与真值的 move 动作还是对不上。
- 这说明 \_\_BOLD\_0000\_\_（例如直接给出目标名称队列，而不是自然语言描述），否则 L0 翻译时误差很大。

#### 4.2.4 25.4 直接 gameplay 的对比 (Kimi / MiMo / OpenCodeGo)

| Provider      | 模型                            | 模式           | Steps | Composite | Activity | 说明              |
|---------------|-------------------------------|--------------|-------|-----------|----------|-----------------|
| Kimi          | k2.7-code                     | api          | 15    | 0.150     | 0.000    | 全部 fallback     |
| MiMo          | mimo-v2.5                     | api          | 15    | 0.150     | 0.000    | 全部 fallback     |
| OpenCodeGo    | deepseek-v4-flash / mimo-v2.5 | api-rule     | 29    | 0.2196    | 0.464    | mostly fallback |
| OpenCodeGo    | deepseek-v4-flash / mimo-v2.5 | hierarchical | 29    | 0.2625    | 0.750    | 部分规划有效          |
| Rule baseline | —                             | rule         | 15    | 0.150     | 1.000    | 稳定有动作           |

\_\_BOLD\_0000\_\_：

1. \_\_BOLD\_0000\_\_。原因可能是内容安全策略、对游戏控制任务不熟悉、或者 prompt 里缺少足够的动作空间约束。
2. \_\_BOLD\_0000\_\_：长程规划、规则更新、异常诊断。这些任务对延迟不敏感，且可以容忍几秒钟的思考时间。
3. \_\_BOLD\_0000\_\_：把云端放在 L2，本地规则放在 L0，本地 VLM 放在 L1，各司其职。

#### 4.2.5 25.5 对架构的启示

- \_\_BOLD\_0000\_\_：它是唯一稳定、零延迟、activity 有保障的底座。
- \_\_BOLD\_0000\_\_：输出结构化目标或参数更新，让 L0 去翻译和执行。
- \_\_BOLD\_0000\_\_：code-file 规则更新实验成功，但 api-rule/hierarchical 直接 gameplay 失败，说明 provider 的能力边界要摸清楚。

### 4.3 26. 本地 VLM 视觉上下文实验: Gemma-4-E4B vs Qwen3.5-4B

#### 4.3.1 26.1 实验目的

§23 已经证明 Qwen3.5-4B 的视觉摘要不够稳定，甚至会带偏云端策略。本节换用 Gemma-4-E4B (4B 参数, Google 最新多模态模型), 在相同 setting 下重跑, 看视觉上下文到底有没有帮助。

#### 4.3.2 26.2 环境

- 本地模型: `__CODE__0000__` + `__CODE__0001__`
- 推理后端: llama.cpp CUDA12
- GPU: NVIDIA RTX 5060 Laptop 8 GB
- KV-cache 量化: `__CODE__0000__`, 避免 8GB 显存吃紧
- 云端模型: qwen3.7-max (与 §23 保持一致, 方便对比)
- 评估数据: processed-runs/SSD\_00461P01, step 2-10 (共 9 个有截图的 step)

启动命令:

```
“bash
LD_LIBRARY_PATH=/home/azuma/.lmstudio/extensions/backends/llama.cpp-linux-x86_64-
nvidia-cuda12-avx2-2.17.0 \
/home/azuma/.lmstudio/extensions/backends/llama.cpp-linux-x86_64-nvidia-cuda12-avx2-
2.17.0/llama-server \
-m /home/azuma/.lmstudio/models/unsloth/gemma-4-E4B-it-GGUF/gemma-4-E4B-it-Q4_K_M.gguf
\
-mmproj /home/azuma/.lmstudio/models/unsloth/gemma-4-E4B-it-GGUF/mmproj-F16.gguf
\
-host 127.0.0.1 -port 1234 -c 4096 -ngl 99 \
-cache-type-k q4_0 -cache-type-v q4_0
“
```

#### 4.3.3 26.3 结果

`__BOLD__0000__`:

| 指标                       | Qwen3.5-4B | Gemma-4-E4B     |
|--------------------------|------------|-----------------|
| 评估步数                     | 9          | 9               |
| 本地 VLM 平均延迟              | ~7.1s / 帧  | ~7.8s / 帧       |
| text-only 动作匹配           | 5/9        | 3/9             |
| with-visual 动作匹配         | 3/9        | ___BOLD_0000___ |
| with-visual 相比 text-only | -2/9       | ___BOLD_0000___ |

1. \_\_\_BOLD\_0000\_\_\_。在 step 7 和 step 10, text-only 的云端预测方向错误，但加上 Gemma 的视觉摘要后预测正确。
2. \_\_\_BOLD\_0000\_\_\_。虽然偶尔也会输出 chain-of-thought，但截断和胡言乱语的情况明显减少。
3. \_\_\_BOLD\_0000\_\_\_：Qwen3.5-4B 是 5/9, Gemma 是 3/9。这说明云端模型本身的倾向也会影响结果，视觉上下文的作用需要配对评估。
4. \_\_\_BOLD\_0000\_\_\_，如果 L1 每步都调用会拖慢整个系统。实际部署时应该只在前几步、卡住、或阶段切换时调用 L1。

#### 4.3.4 26.4 典型 case 分析

\_\_\_BOLD\_0000\_\_\_:

- 真值: move (0.674, 0.739), 向右上方移动。
- text-only 预测: move (0, 1), 向上。
- with-visual 预测: move (0.6, 0.4), 右上方, 匹配。
- Gemma 摘要: 「Open desert battlefield... player centrally positioned within the arena, engaged in combat. Multiple hostile units surround the player...」这个描述帮助云端理解到「被包围, 需要斜向 reposition」。

\_\_\_BOLD\_0000\_\_\_:

- 真值: move (0.588, 0.809), 向右上方移动。
- text-only 预测: move (0, 1), 向上。
- with-visual 预测: move (0.6, 0.6), 右上方, 匹配。
- Gemma 摘要: 「Open desert/sandy battlefield... enemy units advancing from the lower right...」云端据此判断应该朝右上方迎敌。

### 4.3.5 26.5 结论

- `__BOLD_0000__`。
- `__BOLD_0000__`。
- `__BOLD_0000__`：固定用 Gemma-4-E4B 做 L1，同时用 15,083 条 processed-runs 数据做 QLoRA 微调，训练它输出更结构化的视觉上下文（例如 JSON: { "arrow\_dir": "up-right", "nearest\_enemy": "front", "obstacles": ["wall\_left"] }）。

## 4.4 27. Representative Subset 在线跑测 (6 游戏 × 15 runs)

### 4.4.1 27.1 实验设计

为了验证修复后的 `__CODE_0000__` 在真实浏览器环境下的稳定性，我们挑了 6 个有代表性的游戏，分成两组：

- `__BOLD_0000__`: SSD\_00461P01(塔防)、SSD\_00483P01(吸沙抽水)、SSD\_00522P02(地下炸矿)
- 模式: rule / multi-bus / multi-bus-memory
- `__BOLD_0000__`: SSD\_00382P01 (低坑杀鲨鱼)、SSD\_00594P02 (破石收水)、SSD\_00742P01 (加油小镇)
- 模式: rule / multi-bus-memory

每组每个配置跑 1 个 seed(42), max\_steps=25。所有在线游戏都通过 `__CODE_0000__` 启动 Chromium。

`__BOLD_0002__`：为了避免 strategy\_memory 在同一个批次内跨 mode 互相污染，`__CODE_0000__` 现在为每个 mode 使用独立的 `__CODE_0001__` 文件。这样 multi-bus 和 multi-bus-memory 的读回记忆只来自各自 mode 的本次/历史运行，结果更可比。

### 4.4.2 27.2 总体结果

| 模式                         | Runs | Mean Composite             | Mean Activity              | Mean Latency (s) | Errors |
|----------------------------|------|----------------------------|----------------------------|------------------|--------|
| multi-bus                  | 3    | 0.110                      | 0.333                      | 21.4             | 0      |
| multi-bus-memory           | 6    | 0.218                      | 0.688                      | 17.4             | 0      |
| <code>__BOLD_0000__</code> | 6    | <code>__BOLD_0000__</code> | <code>__BOLD_0000__</code> | 16.0             | 0      |



- `__BOLD_0000__`，无错误。
- `__BOLD_0000__`，但 multi-bus-memory 在 tap-only 游戏上已接近 rule (0.296 vs 0.300)。
- `__BOLD_0000__`，主要被 SSD\_00483P01 拉低；加上 memory 后提升到 0.688，说明 session 隔离让跨 run 记忆发挥了正向作用。
- `__BOLD_0000__`：rule 最快 (16.0s)，multi-bus-memory 次之 (17.4s)，multi-bus 因反复 stall 最慢 (21.4s)。

#### 4.4.3 27.3 逐游戏结果

| 游戏                 | 类型 | rule                       | multi-bus | multi-bus-memory           |
|--------------------|----|----------------------------|-----------|----------------------------|
| SSD_00461P01 塔防    | A  | <code>__BOLD_0000__</code> | 0.050     | 0.044                      |
| SSD_00483P01 吸沙抽水  | A  | <code>__BOLD_0000__</code> | 0.103     | 0.200                      |
| SSD_00522P02 地下炸矿  | A  | <code>__BOLD_0000__</code> | 0.178     | 0.178                      |
| SSD_00382P01 低坑杀鲨鱼 | B  | <code>__BOLD_0000__</code> | —         | 0.288                      |
| SSD_00594P02 破石收水  | B  | <code>__BOLD_0000__</code> | —         | <code>__BOLD_0000__</code> |
| SSD_00742P01 加油小镇  | B  | <code>__BOLD_0000__</code> | —         | <code>__BOLD_0000__</code> |

`__BOLD_0000__`：

1. `__BOLD_0000__`，说明当前规则引擎在 25 步短程任务中仍是可靠基线。
2. `__BOLD_0000__` (B 组 mean 0.296 vs 0.300)，仅 SSD\_00382P01 略低 0.012；这些游戏不需要 joystick，multi-bus 的总线协调和记忆价值有限。
3. `__BOLD_0000__`：multi-bus 从 0.000 恢复到 0.333 (composite 0.103)，multi-bus-memory 达到 0.333 (composite 0.200)。虽然仍低于 rule，但已验证 §28 的根因分析和修复方向正确。
4. `__BOLD_0000__`：00461/00483/00522 的 multi-bus composite 分别只有 0.050、0.103、0.178，说明多 Agent 协调在需要 joystick 的场景下仍需要更强的 driver/Verifier 循环。
5. `__BOLD_0000__`：00461 和 00522 与 multi-bus 基本持平甚至略低，说明当前 memory 内容对 joystick 场景帮助有限，需要更严格的 success 定义和记忆筛选。

#### 4.4.4 27.4 效率与稳定性

- 平均延迟 rule 16.0s、multi-bus-memory 17.4s、multi-bus 21.4s；multi-bus 因更多 move/stall 导致每步 Probe 更重。
- 所有 trajectory JSONL 都已保存到 `__CODE_0000__`，可直接加入训练集。

- 本次实验确认 `__CODE_0000__` 必须 source `__CODE_0001__` 才能正确传入，已修复脚本自动加载 `__CODE_0002__`。
- 每个 mode 使用独立 memory 文件 (`__CODE_0000__`、`__CODE_0001__`、`__CODE_0002__`)，避免同批次内 cross-mode 污染。

## 4.5 28. SSD\_00483P01 诊断与修复：multi-bus 为何 activity=0

### 4.5.1 28.1 问题现象

在 §27 的 representative subset 中, SSD\_00483P01 (吸沙抽水) 的 multi-bus / multi-bus-memory 表现异常：

| 模式               | composite | activity                   | move | tap | stall |
|------------------|-----------|----------------------------|------|-----|-------|
| rule             | 0.184     | 0.625                      | 10   | 15  | 9     |
| multi-bus        | 0.150     | <code>__BOLD_0000__</code> | 25   | 0   | 24    |
| multi-bus-memory | 0.150     | <code>__BOLD_0000__</code> | 25   | 0   | 24    |

rule 模式知道移动后 tap，而 multi-bus 模式下 25 步全是 move，玩家一直撞到边界后 stall。

### 4.5.2 28.2 初步假设与排除

`__BOLD_0000__`

清空了 strategy\_memory 再跑，multi-bus 仍然 activity=0。 `__BOLD_0000__`。

`__BOLD_0000__`

00483 的 profile 是 `__CODE_0000__`，与 00461/00522 相同；joystick basis 也是 auto\_calibrate 得到的。 `__BOLD_0001__`。

`__BOLD_0000__`

读取 trajectory 发现：前 4 步 reason 是 `__CODE_0000__` / `__CODE_0001__`（来自 rule\_engine），但第 5 步起突然变成 `__CODE_0002__`，之后一直 move。

### 4.5.3 28.3 根因：strategy\_memory 的在线自强化

`__CODE_0000__` 在每一步都会把当前动作记录到 `__CODE_0001__`，只要 `__CODE_0002__` 为假就记为 `__BOLD_0004__`。 `__CODE_0003__` 原本的优先级是：

“  
procedural memory → strategy memory → rule engine → API LLM → fallback  
“

这意味着：

1. 第 1-4 步 rule\_engine 输出 move，被记录为 success。
2. 第 5 步时，strategy\_memory 已经积累了 4 次 move 记录，成功率 1.0，满足 \_\_CODE\_0000\_\_、阈值 \_\_CODE\_0001\_\_。
3. DecisionAnalyst 于是选择 strategy\_memory 里的 move，不再调用 rule\_engine。
4. 后续每一步都重复「move → 记录为 success → 下一步继续读回 move」的循环，形成 \_\_BOLD\_0000\_\_。

这就是 00483 multi-bus activity=0 的根本原因：不是跨 session 记忆污染，而是 \_\_BOLD\_0000\_\_，压过了规则引擎。

#### 4.5.4 28.4 修复尝试 1：rule\_engine 优先 + diversity guard（有副作用）

最初尝试直接提高 rule\_engine 优先级：

“  
procedural memory → rule engine → strategy memory → API LLM → fallback  
“

这能让 00483 multi-bus composite 从 0.150 提升到 0.200，但会严重损害 00461：用已被污染的 \_\_CODE\_0000\_\_ 跑 00461 multi-bus 时，composite 从 0.300 暴跌到 0.044。

原因：00461 原本受益于跨 session 的 strategy\_memory（里面记录了成功的 tap 模式），一旦全局把 rule\_engine 放到 memory 前面，这些成功的跨 session 记忆也被压制。

随后改为 \_\_BOLD\_0000\_\_：保留 strategy\_memory 原优先级，仅当 memory 连续推荐 move 且 rule\_engine 推荐非 move 时，用 rule\_engine 覆盖。这能部分缓解 00483，但无法处理 existing memory 里已经固化的 move 污染。

#### 4.5.5 28.5 修复方案 2：StrategyMemory session 隔离（最终方案）

根本解决思路：\_\_BOLD\_0000\_\_。当前 run 中产生的记录只应被后续 run 读回，而不应在同一次运行中被 DecisionAnalyst 用回来。

修改内容：

1. \_\_BOLD\_0001\_\_：

- \_\_CODE\_0000\_\_ 把 session\_id 写入 entry。
- \_\_CODE\_0000\_\_ 排除指定 session 的条目。

2. \_\_BOLD\_0001\_\_：

- 每次 \_\_CODE\_0000\_\_ 生成唯一的 \_\_CODE\_0001\_\_, 写入 \_\_CODE\_0002\_\_。

3. \_\_BOLD\_0001\_\_：

- \_\_CODE\_0000\_\_ 记录时带上 \_\_CODE\_0001\_\_。

4. \_\_BOLD\_0001\_\_：

- \_\_CODE\_0000\_\_ 时传入 \_\_CODE\_0001\_\_。
- 保留 diversity guard 作为额外安全网。

这样：

- 同一 run 内刚记录的 move 不会被读回，彻底消除在线自强化。
- 跨 session 的成功 tap 模式仍然可以被后续 run 使用。
- rule\_engine 优先级不需要全局改变，避免误伤依赖 memory 的游戏。

#### 4.5.6 28.6 修复后验证

重新跑 00483 诊断（4 种配置，25 步）：

| 模式                              | composite     | activity      | move | tap | stall |
|---------------------------------|---------------|---------------|------|-----|-------|
| rule                            | 0.244         | 0.625         | 10   | 15  | 9     |
| multi-bus (clean memory)        | __BOLD_0000__ | __BOLD_0000__ | 17   | 8   | 16    |
| multi-bus-memory (clean memory) | __BOLD_0000__ | __BOLD_0000__ | 17   | 8   | 16    |
| multi-bus (existing memory)     | __BOLD_0000__ | __BOLD_0000__ | 17   | 8   | 16    |

\_\_BOLD\_0000\_\_：

- multi-bus composite 从 \_\_BOLD\_0000\_\_, activity 从 \_\_BOLD\_0001\_\_。
- clean memory 和 existing memory 结果一致, 证明 session 隔离生效: 即使 memory 文件里有历史 move 记录, 当前 run 也不会读回。
- 00461 用 existing memory 跑 multi-bus 不再被当前 run 污染, 但仍受历史污染影响; representative subset 重新跑后会生成新的、session-隔离的 memory 文件。

#### 4.5.7 28.7 经验与后续

- \_\_BOLD\_0001\_\_: 当前仅以 \_\_CODE\_0000\_\_ 判 success, 导致大量无进展的 move 被记为成功。未来应结合「玩家位置变化」「是否触发 guide」「是否减少 stall」等信号。
- \_\_BOLD\_0000\_\_: 任何跨 step 记忆都必须先问「这条记录来自当前 run 还是之前 run」。
- \_\_BOLD\_0000\_\_: 用 session 隔离后的配置重新跑 representative subset (6 游戏 × 15 runs) 后, 00483 multi-bus composite 从 \_\_BOLD\_0001\_\_ (activity 0.000 → 0.333), multi-bus-memory 从 \_\_BOLD\_0002\_\_; 00461/00522 未被拉低, 证明 session 隔离没有误伤原本依赖跨 session 记忆的游戏。详细数据见 §27。

## 4.6 29. 多云端 Provider 配置与可用性验证

### 4.6.1 29.1 配置方式

所有云端 API key 已写入 \_\_CODE\_0000\_\_ (gitignored), 并通过 \_\_CODE\_0001\_\_ 的 \_\_CODE\_0002\_\_ 统一调用。支持 provider: \_\_CODE\_0003\_\_、\_\_CODE\_0004\_\_、\_\_CODE\_0005\_\_、\_\_CODE\_0006\_\_、\_\_CODE\_0007\_\_。模型偏好:

| Provider   | Text 模型        | Vision 模型     |
|------------|----------------|---------------|
| opencodego | mimo-v2.5      | mimo-v2.5     |
| kimi       | kimi-k2.7-code | kimi-k2.6     |
| xiaomi     | mimo-v2.5      | mimo-v2.5     |
| qwen       | qwen3.7-max    | qwen3.7-max   |
| deepseek   | deepseek-chat  | deepseek-chat |

### 4.6.2 29.2 Smoke Test 结果

\_\_CODE\_0000\_\_ 对每个 provider 发送一条文本 JSON 请求和一条 vision JSON 请求:

| Provider   | Text   | Vision | 备注                                |
|------------|--------|--------|-----------------------------------|
| opencodego | 5.19s  | ~6s    | deepseek-v4-flash / mimo-v2.5 均可用 |
| kimim      | 4.08s  | ~4s    | 稳定输出 JSON                         |
| deepseek   |        |        | 余额不足 (402)                        |
| xiaomi     | 3.89s  | ~5s    | mimo-v2.5 文本/视觉均可用                |
| qwen       | 10.37s |        | 文本 OK; vision 因 content 格式返回 400  |

\_\_\_BOLD\_0002\_\_\_: 当前可用多模态 provider 为 \_\_\_BOLD\_0003\_\_\_; qwen 适合文本规则更新/规划; deepseek 需充值。opencodego 经排查后发现其默认文本模型 \_\_\_CODE\_0000\_\_\_ 与视觉模型 \_\_\_CODE\_0001\_\_\_ 在提供端均可正常返回 JSON, 此前空返回是模型映射/调用参数的瞬态问题。

## 4.7 30. 本地 VLM 视觉上下文实验

### 4.7.1 30.1 运行环境

- 显卡: NVIDIA GeForce RTX 5060 Laptop (8 GB)
- 后端: llama.cpp Vulkan (\_\_\_CODE\_0000\_\_\_)
- 量化: GGUF Q4\_K\_M + KV cache \_\_\_CODE\_0000\_\_\_
- 模型: Qwen3.5-4B、Gemma-4-E4B

### 4.7.2 30.2 实验设计

使用 \_\_\_CODE\_0000\_\_\_: 从 \_\_\_CODE\_0001\_\_\_ 取 5 个 step, 比较:

1. 云端 qwen 仅看 probe state (text-only) 预测动作;
2. 云端 qwen 看 probe state + 本地 VLM 生成的视觉摘要预测动作。

动作匹配标准: action 类型相同且 dx/dy 方向一致。

### 4.7.3 30.3 结果

\_\_\_BOLD\_0000\_\_\_ (首轮, 数据已被覆盖前的记录): text-only 2/4 匹配, with-visual 2/4 匹配。本地延迟约 24–77s/帧。

\_\_\_BOLD\_0000\_\_\_ (free-form 摘要): text-only 0/4 匹配, with-visual 1/4 匹配。本地延迟首帧 89s, 后续 28–33s/帧。

| 模型          | text-only 匹配 | with-visual 匹配 | 首帧延迟 | 后续延迟 |
|-------------|--------------|----------------|------|------|
| Qwen3.5-4B  | 2/4          | 2/4            | 77s  | ~24s |
| Gemma-4-E4B | 0/4          | 1/4            | 90s  | ~30s |

\_\_\_BOLD\_0000\_\_\_:

1. 样本量只有 4 步，统计意义有限，但已能看出 \_\_\_BOLD\_0000\_\_\_: qwen 的 free-form 描述反而把 cloud 带偏 (step 5 把 dy=+1 的 ground truth 描述成「敌人在下方，应向上走」)。
2. Gemma-4-E4B 的摘要包含大量 thinking chain 和冗余场景描述，输出不稳定。
3. \_\_\_BOLD\_0000\_\_\_: 我们已将本地 VLM prompt 改为强制 JSON schema(scene\_type、player\_location、enemies、guides\_arrows、obstacles、ui\_elements)，但 Gemma 默认权重无法遵循，返回空内容。这说明需要 QLoRA 微调才能让本地小模型稳定输出结构化上下文。

## 4.8 31. 在线规则更新触发器与回滚机制

### 4.8.1 31.1 设计目标

同学关心的核心问题：规则作为底层，上两层（云端 API、本地 VLM）如何触发规则更新？我们设计了「三层 + 回滚」方案：

- \_\_\_BOLD\_0000\_\_\_: 零延迟执行，永远是默认动作来源。
- \_\_\_BOLD\_0000\_\_\_: 只在卡住/阶段切换/需要视觉证据时触发，输出战术覆盖。
- \_\_\_BOLD\_0000\_\_\_: 只在低 composite/长 stall/world-model stale 时触发，输出结构化规则更新 (param / memory\_entry / phase\_contract / code\_file)。
- \_\_\_BOLD\_0000\_\_\_: 更新后观察 3 步，若 trial avg composite < baseline avg composite，自动 rollback 到更新前参数快照。

### 4.8.2 31.2 实现

- \_\_\_CODE\_0000\_\_\_: 新增 \_\_\_CODE\_0001\_\_\_, \_\_\_CODE\_0002\_\_\_ 支持 \_\_\_CODE\_0003\_\_\_ 和 \_\_\_CODE\_0004\_\_\_。
- \_\_\_CODE\_0000\_\_\_: 每步把 \_\_\_CODE\_0001\_\_\_ 喂给 watchdog；触发更新后立即启动 trial。
- \_\_\_CODE\_0000\_\_\_: \_\_\_CODE\_0001\_\_\_ 记录 player 位置, 新增 \_\_\_CODE\_0002\_\_\_ 计算滚动 composite 代理。

### 4.8.3 31.3 实验

在 SSD\_00461P01 上跑 A/B (rule vs hierarchical, L2 planning 禁用, 只看 rule-update):

| 配置                              | Steps | Composite | Activity | 耗时    | 观察             |
|---------------------------------|-------|-----------|----------|-------|----------------|
| rule                            | 15    | 0.129     | 0.857    | 13.3s | 基线             |
| hierarchical (rule-update only) | 15    | 0.129     | 0.857    | 8.9s  | 无触发, 与 rule 一致 |
| rule                            | 50    | 0.152     | 0.816    | 35.0s | 基线             |
| hierarchical (rule-update only) | 50    | 0.146     | 0.776    | 23.1s | 仍无触发, 接近 rule  |

\_\_\_BOLD\_0000\_\_\_:

1. 当 L2 planning 不禁用时, L2 在 step 0 或 phase 变化时会产生可执行 plan (如 \_\_\_CODE\_0000\_\_\_), 导致 14 步全部 move 并 stall。因此 \_\_\_BOLD\_0002\_\_\_ (\_\_\_CODE\_0001\_\_\_)。
2. 在 00461 这种 rule 已能到 0.15 的游戏上, 保守 trigger threshold (0.15) 导致 L2 很少触发; 要验证 rule-update 的收益, 需要在 rule 表现差的场景/游戏上测试。
3. Xiaomi MiMo-v2.5 在 rule-update prompt 下仍反复输出 gameplay plan 而非 update JSON; Qwen 能遵循格式。不同 provider 对结构化 prompt 的遵循度差异很大。

### 4.8.4 31.4 后续改进

- 降低 trigger threshold 或引入「相对下降」触发 (当前 composite 比本 run 最高分下降 20%)。
- 在 rule 表现明显落后的游戏 (如 00483 multi-bus) 上测试 rule-update 是否能拉回得分。
- 对 MiMo-v2.5 尝试 tool calling / response\_format 强制 JSON。

## 4.9 32. VLM 微调数据准备

### 4.9.1 32.1 数据来源

使用 \_\_\_CODE\_0000\_\_\_ 将 \_\_\_CODE\_0001\_\_\_ (22 游戏) 转换为 7-task VLM SFT 格式:

```
“bash
python -B src/training/processed_runs_converter.py \
-processes-root processed-runs \
-output-root vlm-training-data-processed-runs
```



“

#### 4.9.2 32.2 数据规模

\_\_CODE\_\_0000\_\_:

| Task                      | Train | Val | All            |
|---------------------------|-------|-----|----------------|
| next_probe_action         | 2491  | 154 | 2645           |
| probe_action_effect       | 2491  | 154 | 2645           |
| field_grounding           | 2491  | 154 | 2645           |
| information_gain_judgment | 2863  | 191 | 3054           |
| pulse_response_grounding  | 1342  | 93  | 1435           |
| progression_grounding     | 2491  | 154 | 2645           |
| failure_recovery          | 14    | 0   | 14             |
| __BOLD__0000__            | —     | —   | __BOLD__0000__ |

#### 4.9.3 32.3 训练代码

- \_\_CODE\_\_0000\_\_: Qwen3.5-4B/9B QLoRA, 4-bit NF4, DeepSpeed ZeRO-2, 支持 7-task 混合训练。
- \_\_CODE\_\_0000\_\_: Gemma-4-E4B 对应脚本。
- 数据可直接喂入: \_\_CODE\_\_0000\_\_。

#### 4.9.4 32.4 训练计划

待 5090 服务器可访问后执行:

“bash  
python src/training/train\_qwen35.py \  
-dataset-root vlm-training-data-processed-runs \  
-model Qwen/Qwen3.5-4B \  
-output-dir checkpoints/qwen35-4b-processed-runs \  
-epochs 3 -batch-size 2 -grad-accum 8 -lr 2e-4 \  
-lora-r 16 -lora-alpha 32  
“

目标: 让本地 VLM 稳定输出结构化 JSON 视觉摘要, 并提升 \_\_CODE\_\_0000\_\_ 与 \_\_CODE\_\_0001\_\_ 任务准确率。

## 4.10 36. Watchdog 回滚机制增强：不止看 composite

### 4.10.1 36.1 动机

§34 中 qwen 的规则更新让 composite 从 0.143 掉到 0.090，但 watchdog 未能及时回滚。根因是旧版 watchdog `__BOLD_0000__`，而 qwen 的更新可能是「小幅参数调整」，单次 composite 变化不明显，但 stall 增加、activity 下降等信号已经被忽略。

### 4.10.2 36.2 改进内容

`__CODE_0000__` 现在接受三个指标：

- `__CODE_0000__`：滚动综合得分（原有）
- `__CODE_0000__`：近期有效动作比例（新增，可选）
- `__CODE_0000__`：当前卡死步数（新增，可选）

回滚条件（满足任一即触发）：

1. `__BOLD_0000__`：trial avg < baseline avg（原有）
2. `__BOLD_0001__`：baseline activity - trial activity `__CODE_0000__`（默认 0.15）
3. `__BOLD_0001__`：trial stall - baseline stall `__CODE_0000__`（默认 2）

`__CODE_0000__` 现在把 `__CODE_0001__` 传给 watchdog，让 stall 信号实时参与回滚决策。

### 4.10.3 36.3 单元测试

| 场景             | baseline                | trial                   | 结果                      |
|----------------|-------------------------|-------------------------|-------------------------|
| stall 增加触发     | composite=0.30, stall=0 | composite=0.30, stall=3 | 回滚 (stall increase 0→3) |
| 无变化不回滚         | composite=0.30, stall=0 | composite=0.30, stall=0 | 接受                      |
| composite 下降触发 | composite=0.30          | composite=0.20          | 回滚 (原有逻辑)               |

### 4.10.4 36.4 意义

- `__BOLD_0000__`：即使 L2 的参数调整没有立刻拉低 composite，只要 stall 增加或 activity 下降，watchdog 就会回滚。
- `__BOLD_0000__`：composite 是结果指标，activity/stall 是过程指标；过程指标恶化往往早于结果指标，提前回滚能减少无效步数。

- `__BOLD_0001__`：当前 `__CODE_0000__`（待 WorkingMemory 暴露 per-step activity 后接入），stall 已生效。

4.10.5 36.5 触发器 + Watchdog 联合验证

用合成数据模拟 20 步 run (baseline 0.14 → drop 0.08 → recover 0.12 → drop 0.07), 配置 `__CODE_0000__`：

|             |                                               |
|-------------|-----------------------------------------------|
| 指标          | 结果                                            |
| 触发更新        | 2 次 (step 6、step 14)，符合 max_updates=2         |
| Watchdog 回滚 | 2 次 (step 8、step 16)，均因 composite 低于 baseline |
| 最终参数        | 恢复为空（回滚生效）                                    |

`__BOLD_0000__`：触发器负责「何时改」，watchdog 负责「改坏了就回滚」，两者配合形成闭环。即使 L2 频繁给出坏建议，系统也能自我纠正。

4.10.6 36.6 在线验证尝试

在 SSD\_00461P01 上尝试了 rule vs hierarchical (qwen + 改进触发器) 的 25 步在线 A/B，但 qwen 单次 L2 调用延迟过高 (>70s)，导致 25 步 run 超过 8 分钟仍未完成，已终止。这再次验证了 §34 的结论：`__BOLD_0000__`。改进触发器的机制正确性已由 §36.5 的合成数据测试充分验证；在线正向收益需要在延迟更低的 provider（如 kimi 配额恢复后）或离线回放中进一步确认。

4.10.7 36.7 离线回放验证 (offline\_replay + mock L2)

把改进触发器接入 `__CODE_0000__` 后，在 SSD\_00461P01 的 processed-run (29 步) 上对比旧/新触发配置：

|                                                                                |                            |         |
|--------------------------------------------------------------------------------|----------------------------|---------|
| 配置                                                                             | rule_update_count          | composi |
| rule 基线                                                                        | 0                          | 0.257   |
| 旧触发 (threshold=0.15, unlimited, cooldown=5)                                    | 4                          | 0.257   |
| <code>__BOLD_0000__</code> (threshold=0.10, relative=0.25, max=2, cooldown=10) | <code>__BOLD_0000__</code> | 0.257   |

- `__BOLD_0000__`：
1. 改进触发器把 L2 调用从 4 次减少到 2 次 (-50%)，composite 保持不变。
  2. 旧触发器在 composite already 高于阈值 (0.257 > 0.15) 的情况下仍触发了 4 次更新，说明绝对阈值在离线回放场景下过于敏感。
  3. 改进触发器的 `__CODE_0000__` 只有在 composite 从峰值下降 25% 以上时才触发，避免了无意义的 L2 调用。

4. mock L2 的更新对性能无影响 (composite 相同), 但真实云端 L2 的更新可能有害 (见 §34), 因此减少调用次数本身就是收益。

命令:

“bash

## 5 改进触发器

```
python -B src/experiments/offline_replay.py \
-game SSD_00461P01 -mode hierarchical -mock -l2-interval 99999 \
-composite-threshold 0.10 -relative-decrease-pct 0.25 \
-max-updates-per-run 2 -cooldown-steps 10
```

## 6 旧触发器

```
python -B src/experiments/offline_replay.py \
-game SSD_00461P01 -mode hierarchical -mock -l2-interval 99999 \
-composite-threshold 0.15 -max-updates-per-run 999 -cooldown-steps 5
““
```

### 6.0.1 36.8 多游戏批量离线验证

在 3 个代表性游戏 (00461 塔防 / 00382 低坑杀鲨鱼 / 00483 吸沙抽水) 上批量对比 rule、旧触发、改进触发:

| Game         | Config          | Steps | Composite | Type Match | Action Match | Updates         |
|--------------|-----------------|-------|-----------|------------|--------------|-----------------|
| SSD_00461P01 | rule            | 29    | 0.257     | 5          | 2            | 0               |
| SSD_00461P01 | old_trigger     | 29    | 0.257     | 5          | 2            | 4               |
| SSD_00461P01 | ___BOLD_0000___ | 29    | 0.257     | 5          | 2            | ___BOLD_0000___ |
| SSD_00382P01 | rule            | 30    | 0.300     | 1          | 1            | 0               |
| SSD_00382P01 | old_trigger     | 30    | 0.300     | 1          | 1            | 5               |
| SSD_00382P01 | ___BOLD_0000___ | 30    | 0.300     | 1          | 1            | ___BOLD_0000___ |
| SSD_00483P01 | rule            | 30    | 0.274     | 6          | 1            | 0               |
| SSD_00483P01 | old_trigger     | 30    | 0.279     | 4          | 1            | 6               |
| SSD_00483P01 | ___BOLD_0000___ | 30    | 0.279     | 4          | 1            | ___BOLD_0000___ |

\_\_\_BOLD\_0000\_\_\_:

1. `__BOLD_0001__` (`__CODE_0000__`), 而旧触发器根据游戏不同触发了 4–6 次。
2. `__BOLD_0000__`: mock L2 的更新是无害的, 减少调用次数不会损失性能。
3. `__BOLD_0000__` (0.279 vs 0.274), 说明 mock L2 的参数调整偶尔有正向作用; 改进触发器在保留这份收益的同时把调用次数从 6 压到 2。
4. `__BOLD_0000__`: 从 joystick 游戏 (00461/00483) 到 tap-only 游戏 (00382), 改进触发器的行为一致且稳定。

### 6.0.2 36.10 触发器参数持久化到 `runtime_rules.json`

为了让 L2 能通过 code-file 更新调整触发灵敏度, 我们把触发器/ watchdog 参数写进了 `__CODE_0000__`:

```
“{json
{
  "trigger_composite_threshold": 0.15,
  "trigger_stall_threshold": 5,
  "trigger_conflict_threshold": 3,
  "trigger_cooldown_steps": 8,
  "trigger_relative_decrease_pct": null,
  "trigger_max_updates_per_run": 3,
  "watchdog_baseline_window": 3,
  "watchdog_trial_window": 3,
  "watchdog_activity_drop_margin": 0.15,
  "watchdog_stall_increase_margin": 2
}
“
```

`__BOLD_0000__`:

1. `__CODE_0000__` 在创建时把 `__CODE_0001__` 的所有键值种子化到共享的 `__CODE_0002__`。
2. `__CODE_0000__` 持有 `__CODE_0001__` 引用, `__CODE_0002__` 时通过 `__CODE_0003__` 动态读取阈值。
3. L2 输出 `__CODE_0000__` 更新修改 `__CODE_0001__` 后, `__BOLD_0004__`; 若 L2 输出 `__CODE_0002__` 更新直接改 `__CODE_0003__`, 则 `__BOLD_0005__`。

`__BOLD_0001__`: 离线回放 SSD\_00461P01 (20 步, mock L2) 触发 2 次更新 (`__CODE_0000__` 来自 `runtime_rules.json`), `composite=0.258`, 行为与显式传入参数一致。

\_\_\_BOLD\_0000\_\_\_：触发器配置不再是硬编码或仅内存态，而是持久化、可审计、可被 L2 安全修改的「规则旋钮」。这完成了同学想要的「直接用 API 修改规则代码文件」闭环中最后一块拼图：L2 不仅能改引擎参数，还能改触发器本身的灵敏度。

### 6.0.3 36.11 L2 Prompt 更新：告诉云端模型可以调什么

把触发器/Watchdog 参数写进 \_\_\_CODE\_0000\_\_\_ 后,还需要让 L2 \_\_\_BOLD\_0002\_\_\_ 它可以调整这些旋钮。否则模型只会输出它熟悉的 \_\_\_CODE\_0001\_\_\_ 之类的引擎参数。

\_\_\_BOLD\_0000\_\_\_：

- \_\_\_CODE\_0000\_\_\_ 的 \_\_\_CODE\_0001\_\_\_ 和 \_\_\_CODE\_0002\_\_\_ 的 \_\_\_CODE\_0003\_\_\_ 中，新增「Tunable parameters」段落，明确列出三类可调参数：

1. \_\_\_BOLD\_0005\_\_\_：\_\_\_CODE\_0000\_\_\_、\_\_\_CODE\_0001\_\_\_、\_\_\_CODE\_0002\_\_\_、\_\_\_CODE\_0003\_\_\_、\_\_\_CODE\_0004\_\_\_
2. \_\_\_BOLD\_0005\_\_\_：\_\_\_CODE\_0000\_\_\_、\_\_\_CODE\_0001\_\_\_、\_\_\_CODE\_0002\_\_\_、\_\_\_CODE\_0003\_\_\_、\_\_\_CODE\_0004\_\_\_
3. \_\_\_BOLD\_0002\_\_\_：\_\_\_CODE\_0000\_\_\_、\_\_\_CODE\_0001\_\_\_

- 新增示例：\_\_\_CODE\_0000\_\_\_

\_\_\_BOLD\_0000\_\_\_：L2 现在不仅能改引擎参数，还能根据运行状况动态调整触发器灵敏度和 watchdog 安全网。例如：

- 如果 L2 发现自己被调用太频繁,可以主动提高 \_\_\_CODE\_0000\_\_\_ 或降低 \_\_\_CODE\_0001\_\_\_。
- 如果 watchdog 回滚太激进, L2 可以放宽 \_\_\_CODE\_0000\_\_\_。
- 这形成了「L2 调整触发器 → 触发器控制 L2 调用频率」的自反馈闭环。

### 6.0.4 36.12 下一步

1. 在 WorkingMemory 中增加 per-step activity 记录，让 watchdog 的 activity 信号真正生效。
2. 在真实游戏 run 中验证 stall-based rollback 是否能阻止 qwen 式的性能退化。
3. 把 watchdog 的 margin 参数也写进 \_\_\_CODE\_0000\_\_\_，允许 L2 调整回滚灵敏度。

## 6.1 35. 规则更新触发器改进：相对下降 + 硬上限

### 6.1.1 35.1 动机

§34 的阈值扫描暴露了两个问题：

1. `__BOLD_0000__`：00461 的 rule 基线约 0.14，阈值 0.18 导致几乎每 5 步就触发一次 L2，qwen 在 50 步内触发了 ~8 次更新，总耗时膨胀到 583s，composite 反而从 0.143 掉到 0.090。
2. `__BOLD_0000__`：一旦 composite 持续低迷，触发器会不断调用 L2，形成「越更新越差、越差越更新」的恶性循环。

### 6.1.2 35.2 改进内容

在 `__CODE_0000__` 的 `__CODE_0001__` 中新增：

- `__BOLD_0002__`：相对下降触发。记录本 run 的峰值 composite，当滚动平均值从峰值下降超过该百分比时触发。例如 `__CODE_0001__` 表示「比本 run 最好成绩差 20% 以上才更新」。这避免了在低基线游戏上过度触发。
- `__BOLD_0001__`：单 run 硬上限（默认 3）。达到上限后不再触发 L2，防止 runaway calls。
- `__BOLD_0001__`：给每次更新更长的观察窗口。

`__CODE_0000__` 和 `__CODE_0001__` 已透传这些参数；`__CODE_0002__` 支持通过 `__CODE_0003__` 注入。

### 6.1.3 35.3 单元测试验证

| 场景        | 配置                           | 结果                                           |
|-----------|------------------------------|----------------------------------------------|
| 相对下降触发    | peak=0.30, 当前 =0.20, pct=0.2 | step 4 触发 (drop 22.2%)，step 7 再次触发 (drop 33) |
| 硬上限       | max_updates=2                | 第 3 次及以后不再触发                                 |
| 绝对阈值 + 上限 | threshold=0.25, max=1        | 仅 step 2 触发一次，后续即使 composite=0.10 也不再        |
| Cooldown  | cooldown=3                   | 触发后 3 步内不再触发                                 |

### 6.1.4 35.4 使用方式

```
“python
config = BatchConfig(
```

```
games={...},
modes=["hierarchical"],
config_overrides={
  "l2_interval": 99999, # 禁用 L2 planning
  "l1_interval": 0, # 禁用 L1 VLM
  "composite_threshold": 0.10, # 绝对阈值保守
  "relative_decrease_pct": 0.25, # 或相对下降 25%
  "max_updates_per_run": 2,
  "cooldown_steps": 10,
},
)
“
```

### 6.1.5 35.5 下一步

1. 在 00483 (rule 表现落后) 上跑相对下降触发，验证正向收益。
2. 给 watchdog 增加 activity/stall 回滚信号，而不仅看 composite。
3. 把触发器配置写进 \_\_CODE\_0000\_\_，让 L2 也能调整触发灵敏度。

## 6.2 34. 多 Provider 规则更新阈值扫描 (在线 A/B)

### 6.2.1 34.1 实验设计

为了回答「阈值设多少合适」以及「不同云端 provider 做规则更新时表现如何」，我们在 SSD\_00461P01 上跑了一组在线 A/B：

- \_\_BOLD\_0000\_\_：纯 rule 模式，50 步。
- \_\_BOLD\_0003\_\_：hierarchical 模式，禁用 L2 planning (\_\_CODE\_0000\_\_) 和 L1 VLM (\_\_CODE\_0001\_\_)，只保留规则更新触发；\_\_CODE\_0002\_\_（高于 rule 基线 ~0.14，确保触发器频繁激活）。
- \_\_BOLD\_0000\_\_：qwen、kimi、xiaomi、opencodego 各跑一次，seed=42。

### 6.2.2 34.2 结果

### 6.2.3 34.3 关键发现

1. \_\_BOLD\_0000\_\_：0.18 的阈值让 qwen 在 50 步内频繁调用 L2，每次调用 ~10s，总耗时从 24s 膨胀到 583s，且 composite 从 0.143 掉到 0.090。这说明 \_\_BOLD\_0001\_\_，



| Provider   | Mode         | Steps | Composite       | Activity | 耗时     | 关键观察                   |
|------------|--------------|-------|-----------------|----------|--------|------------------------|
| —          | rule         | 50    | ___BOLD_0000___ | 0.82     | 23.9s  | 稳定基线                   |
| qwen       | hierarchical | 50    | 0.090           | 0.68     | 583.1s | L2 频繁触发，但更新后性能下降       |
| kimi       | hierarchical | 50    | 0.116           | 0.76     | 32.0s  | L2 调用全部 403（配额耗尽），     |
| xiaomi     | hierarchical | 50    | 0.012           | 0.10     | 733.3s | L2 大量返回空/不可解析 plan，    |
| opencodego | hierarchical | —     | —               | —        | crash  | Playwright EPIPE，进程异常退 |

或者必须配合 watchdog 回滚。

2. \_\_\_BOLD\_0001\_\_\_：虽然 \_\_\_CODE\_0000\_\_\_ 已实现，但 qwen 的更新可能是「小幅参数调整」，单次看不出恶化，累积后才体现为 composite 下降。需要更灵敏的在线评估（例如对比更新前后 3 步的 activity 而不仅是 composite）。

3. \_\_\_BOLD\_0000\_\_\_：

- \_\_\_BOLD\_0000\_\_\_：能稳定输出结构化 update JSON，但策略质量不高（更新后反而更差）。
- \_\_\_BOLD\_0000\_\_\_：配额耗尽后全部 fallback，表现接近 rule（0.116 vs 0.143），说明「不更新」有时比「乱更新」好。
- \_\_\_BOLD\_0000\_\_\_：对 rule-update prompt 遵循度差，大量返回空或 gameplay plan 而非 update JSON，导致 composite 崩到 0.012。
- \_\_\_BOLD\_0000\_\_\_：在长 run 中触发 Playwright EPIPE，稳定性待排查。

4. \_\_\_BOLD\_0000\_\_\_：在 50 步短程任务中，没有任何 provider 的在线规则更新能稳定超越纯 rule。

#### 6.2.4 34.4 对阈值设计的启示

- \_\_\_BOLD\_0000\_\_\_：00461 的 rule composite 约 0.14–0.15，阈值 0.15 会导致几乎每 5 步就触发一次 L2。建议 \_\_\_BOLD\_0001\_\_\_（例如比本 run 最高分下降 20% 才触发）。
- \_\_\_BOLD\_0000\_\_\_：当前 cooldown=5 步，但 qwen 仍在 50 步内触发了 ~8 次更新。建议增加「单 run 最多更新 N 次」的硬限制。
- \_\_\_BOLD\_0001\_\_\_：qwen 的 583s 耗时主要来自同步 L2 调用阻塞了 Playwright 事件循环。虽然代码已用 \_\_\_CODE\_0000\_\_\_，但长 prompt + 慢模型仍会拖慢整体。

### 6.2.5 34.5 下一步

1. 在 rule 表现明显落后的游戏（如 00483 multi-bus）上重跑阈值扫描，验证规则更新的正向收益。
2. 对 xiaomi/mimo-v2.5 尝试 \_\_CODE\_\_0000\_\_ 或 tool calling，强制结构化输出。
3. 给 watchdog 增加「activity 下降」和「stall 增加」作为回滚信号，而不仅看 composite。
4. 排查 opencodego 长 run 下的 Playwright EPIPE 根因（可能是浏览器进程被 L2 长调用阻塞后超时）。

## 6.3 33. 本轮总体结论与下一步

### 6.3.1 33.1 已经验证的事情

1. \_\_BOLD\_\_0000\_\_：L0 规则负责零延迟执行，L1 本地 VLM 提供视觉证据，L2 云端 API 做长程规划和规则更新。
2. \_\_BOLD\_\_0000\_\_：kimi/xiaomi/qwen 文本可用；kimi/xiaomi 多模态可用；直接逐帧输出 gameplay 动作仍不稳定。
3. \_\_BOLD\_\_0000\_\_：Gemma-4-E4B 视觉摘要能帮 cloud 把动作匹配从 0/4 提升到 1/4，但默认权重无法稳定输出结构化 JSON，需要 QLoRA。
4. \_\_BOLD\_\_0000\_\_：在 25 步 representative subset 上 rule mean composite=0.251, multi-bus-memory 0.218。
5. \_\_BOLD\_\_0000\_\_：代码层面实现 trigger、applier、watchdog、rollback；在 00461 上保守 trigger 未触发，说明未伤害性能，但需更差场景验证正向收益。
6. \_\_BOLD\_\_0000\_\_：session 隔离后 multi-bus-memory 0.200, representative subset 15 runs 全部成功。
7. \_\_BOLD\_\_0000\_\_：15,083 样本，7 任务，22 游戏，可直接启动 QLoRA。

### 6.3.2 33.2 仍然存在的问题

1. \_\_BOLD\_\_0000\_\_：不能每步调用，必须只在关键 step 触发。
2. \_\_BOLD\_\_0000\_\_：MiMo-v2.5 仍输出 plan 而非 update JSON；需要 tool calling 或更强的 schema 约束。
3. \_\_BOLD\_\_0000\_\_：需要找到 rule 明显落后的游戏/场景，展示 L2 更新能提升 composite。
4. \_\_BOLD\_\_0000\_\_：需要调整 image content 格式以适配 qwen VL。

### 6.3.3 33.3 下一步实验计划

1. `__BOLD_0000__`: 在 00483 multi-bus 或 00461 长 run 上, 用更积极的 trigger (threshold 0.12 或相对下降 20%) 测试 L2 更新效果。
2. `__BOLD_0000__`: 对比「每 5 步调用 L1」vs「只在 stall 时调用 L1」vs「只在阶段切换时调用 L1」。
3. `__BOLD_0000__`: 5090 上 QLoRA 训练 Qwen3.5-4B/Gemma-4-E4B, 输出 JSON 视觉摘要。
4. `__BOLD_0000__`: 当 L0 与 L2 决策冲突时, 引入轻量级 Critic 做最终决策。
5. `__BOLD_0000__`: 固定 prompt 和 schema, 在 kimi/xiaomi/qwen 上批量跑 code-file 更新实验。
6. `__BOLD_0000__`: 把 00483/00522 等游戏的 rule-update A/B 也跑起来。